



High-Performance Embedded Computing in Space: Evaluation of Platforms for Vision-Based Navigation

George Lentaris,^{*} Konstantinos Maragos,[†] Ioannis Stratakis,[‡] Lazaros Papadopoulos,[†] Odysseas Papanikolaou,[†] and Dimitrios Soudris[‡]

National Technical University of Athens, 15780 Athens, Greece

Manolis Lourakis[§] and Xenophon Zabulis[§]

Foundation for Research and Technology—Hellas, 711 10 Heraklion, Greece

David Gonzalez-Arjona[¶]

GMV Aerospace and Defence SAU, 28760 Madrid, Spain

and

Gianluca Furano^{**}

European Space Agency, 2201 AZ Noordwijk, The Netherlands

DOI: 10.2514/1.1010555

Vision-based navigation has become increasingly important in a variety of space applications for enhancing autonomy and dependability. Future missions, such as active debris removal for remediating the low Earth orbit environment, will rely on novel high-performance avionics to support advanced image processing algorithms with substantial workloads. However, when designing new avionics architectures, constraints relating to the use of electronics in space present great challenges, further exacerbated by the need for significantly faster processing compared to conventional space-grade central processing units. With the long-term goal of designing high-performance embedded computers for space, in this paper, an extended study and tradeoff analysis of a diverse set of computing platforms and architectures (i.e., central processing units, multicore digital signal processors, graphics processing units, and field-programmable gate arrays) are performed with radiation-hardened and commercial off-the-shelf technology. Overall, the study involves more than 30 devices and 10 benchmarks, which are selected after exploring the algorithms and specifications required for vision-based navigation. The present analysis combines literature survey and in-house development/testing to derive a sizable consistent picture of all possible solutions. Among others, the results show that certain 28 nm system-on-chip devices perform faster than space-grade and embedded central processing units by 1–3 orders of magnitude, while consuming less than 10 W. Field-programmable gate array platforms provide the highest performance per watt ratio.

I. Introduction

VISION-BASED navigation (VBN) endows robotic systems with the ability to guide themselves autonomously in an unknown environment by using active or passive image sensors to perceive their surroundings. Vision applications in space vary from rover exploration on Mars with simultaneous localization and mapping to moon landing with landmark tracking and hazard avoidance, and to spacecraft proximity operations with pose estimation of cooperative or uncooperative orbiting targets. However, the ever-increasing computational demands of these enabling technologies are now challenging classical onboard computing, both in terms of architecture and processing power. An example where this becomes especially apparent concerns active debris removal (ADR) scenarios, whose aim is to actively remove heavy debris objects by capturing them and then either disposing them by destructive reentry in the atmosphere or moving them to graveyard orbits [1]. In future ADR missions, chaser spacecraft must rely, among others, on high-definition cameras and complex computer vision algorithms performing image feature extraction, matching, three-dimensional (3-D) reconstruction, tracking, etc., at high frame rates. Given the requirements for fast and accurate autonomous relative navigation, the gap between algorithmic complexity and conventional space-grade processors is quickly widening and calls for novel high-performance embedded computing in space.

A. Conventional Avionics Architecture

Classical onboard data handling [2] is based on a central onboard computer (OBC) connected to various platform subsystems, such as the radio communication, attitude and orbit control, payloads, etc. (Fig. 1). Most often, the internal communication network involves a serial bus of high data integrity (i.e., the military standard 1553 or the controller area network, CAN, bus at around 1 Mbps). The OBC has two primary functions, namely to decode telecommands and to serve as the central terminal of the spacecraft. In few cases, the OBC also handles payload data (e.g., for a remote sensing application that would otherwise require a separate processor). The peripheral terminals are standalone units or embedded within the instruments of the spacecraft. The more complicated of these instruments include their own embedded processor to support intelligent control units, sophisticated remote bus interfaces, and specialized science data processing, which normally demands higher performance than that of the OBC [2]. Occasionally, the instrument data are collected by an independent “payload data-handling system” supporting relatively high data rates.

Received 9 April 2017; revision received 17 November 2017; accepted for publication 22 January 2018; published online Open Access 20 February 2018. Copyright © 2018 by the American Institute of Aeronautics and Astronautics, Inc. All rights reserved. All requests for copying and permission to reprint should be submitted to CCC at www.copyright.com; employ the ISSN 2327-3097 (online) to initiate your request. See also AIAA Rights and Permissions www.aiaa.org/randp.

^{*}Research Associate, School of Electrical and Computer Engineering; glentaris@microlab.ntua.gr.

[†]Research Associate, School of Electrical and Computer Engineering.

[‡]Associate Professor, School of Electrical and Computer Engineering; dsoudris@microlab.ntua.gr.

[§]Principal Researcher, Institute of Computer Science, P.O. Box 1385; lourakis@ics.forth.gr.

[¶]Avionics and On-Board Software Engineer, Tres Cantos.

^{**}On-Board Computer Engineer, Microelectronics and Data Systems Division, European Space Technology Centre, Keplerlaan 1; gianluca.furano@esa.int.

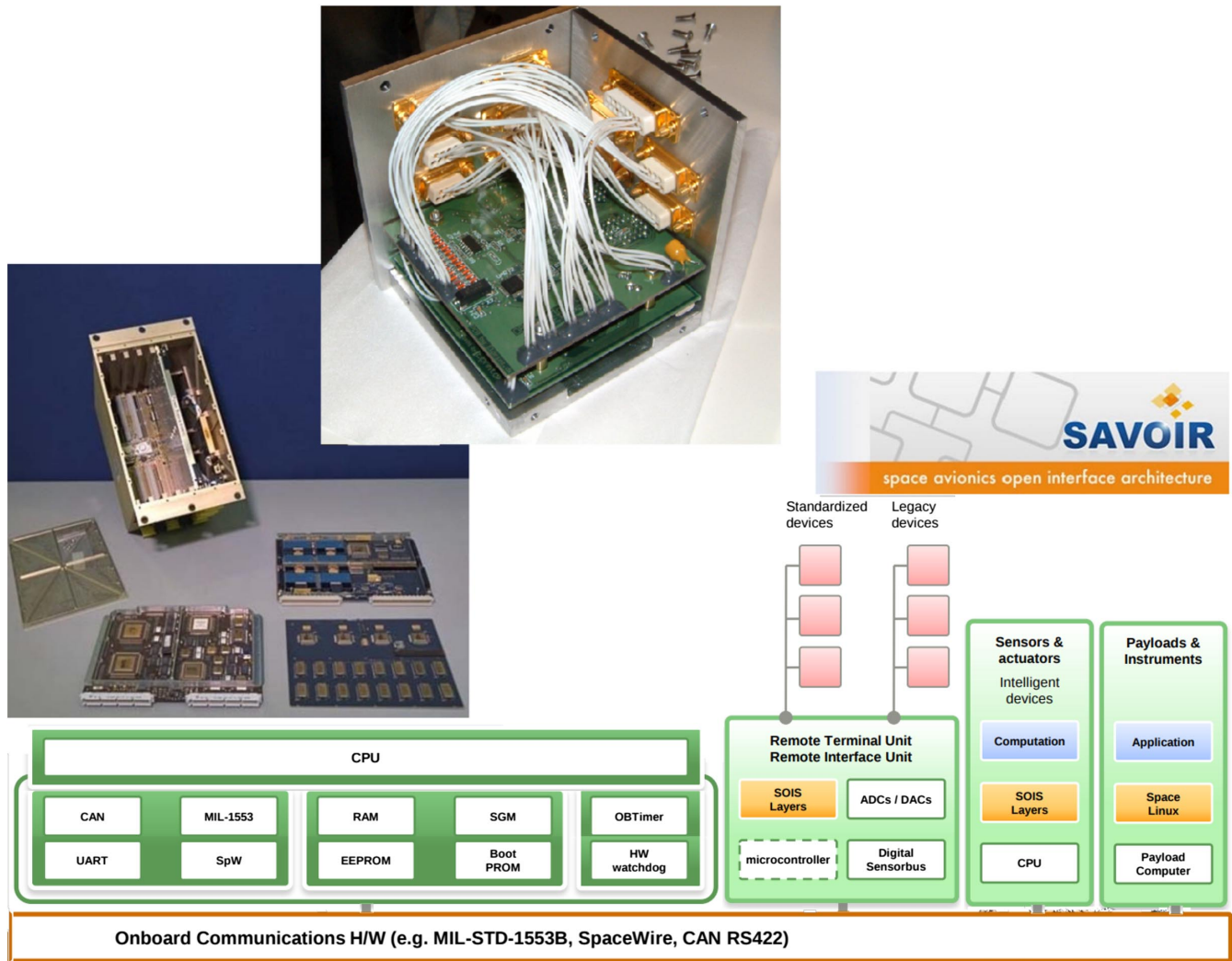


Fig. 1 Examples of OBC (for SSETI Express, top; DMS-R for the International Space Station, left) and avionics architecture (SAVOIR, bottom). Images copyright ESA.

Routine spacecraft tasks such as attitude control, housekeeping reports, command decoding, and systems management require very little processing power, which can be provided even by simple 8-bit microcontrollers [2]. More demanding tasks, such as payload software, intensive digital signal processing (e.g., time/frequency-domain filtering, compression, feature extraction, and matching), data formatting, error control, or encryption, are executed on more advanced microprocessors. Such tasks within payloads/experiments may have a dedicated controller interfacing with the platform, under real-time operation, to output data in source packet format. Their controller will typically be a standard microprocessor, such as the SPARC-based LEON, a digital signal processor (DSP) device, such as the TSC21020F, or, in more extreme cases, even an application-specific integrated circuit (ASIC). Peripheral supporting devices, like universal asynchronous receiver and transmitters, dynamic random-access memory controllers, I/O controllers, and communications devices, have custom implementations on field-programmable technology [2]. Until now, the aforementioned computers have predominantly been of space-grade technology and hence significantly slower compared to their contemporary terrestrial counterparts, lagging behind them by 1–2 orders of magnitude in performance [2–4].

B. Trends and Hazards for Future Avionics

Recent trends in the space industry push the onboard processing power and storage toward terrestrial industrial practices [2]. The goal is to support more complex data organization, autonomous decision making, intensive signal processing and multitasking, and the coordination of large distributed development teams. Commercial-off-the-shelf (COTS) devices are competing with conventional radiation-hardened (rad-hard) components by offering the advantages of higher performance, faster/cheaper development, ease of maintenance, off-the-shelf development systems and processing/communications libraries, built-in file handling, multitasking and intertask communication, and software reliability [2]. Lately, the use of COTS in high-reliability space applications is on the rise and ranges from memory chips in flagship planetary explorers to various parts in pico/nanosats [4]. Operational constellations of the latter currently provide useful services based on COTS that have been selected and tested to meet certain mission requirements [5]. European Space Agency (ESA) roadmaps [6] present an emerging trend toward COTS adoption, while the United States is moving aggressively to accept COTS chips in space. This is also evident in institutional missions (e.g., by ESA and CNES, who have multiple activities with commercial Xilinx Zynq devices for processing).

On the downside, when deployed in space, COTS devices can be severely harmed by the harsh environmental conditions of high vacuum, extreme temperatures, and high levels of ionizing radiation, or even by the vibrations during launch. Space users consider applying mitigation methods (e.g., redundancy) to handle the damage and increase the availability/reliability of these components in flight [3,6]. More specifically [2], although with very low probability, plastic materials may outgas under vacuum or cause electrostatic discharge. Thermal-wise, many COTS devices can only operate in a limited temperature range (e.g., -30 to $+70^{\circ}\text{C}$), which must be moderated within a spacecraft. Radiation-wise, a

non-rad-hard COTS device will suffer from total dose effects and single-event effects (SEEs) owing to the trapped particles in the Van Allen belts surrounding Earth, the galactic cosmic rays originating from deep space, and solar-flare particles [7]. In general [2], avionics should be tolerant to 10–50 krad total ionizing dose (TID) for low Earth orbit (LEO) missions and 100–300 krad for MEO/GEO missions. However, specifically for short-duration missions in LEO orbit (e.g., as in an ADR scenario), the aforementioned radiation effects could be tolerable even for COTS due to the protection offered by the Earth's magnetic field and the limited exposure time.

C. Challenges in the Advent of Vision-Based Navigation and High-Performance Avionics

COTS processing units will facilitate high-performance embedded computing in space, with VBN being one of the first subsystems to reap the increased benefits of new technologies. In turn, advanced image processing systems and complex guidance, navigation, and control (GNC) will become the enabling building blocks in ADR, as well as in similar future space proximity operations such as inspection, in-orbit servicing, cargo and satellite delivery, space tug, in-orbit structure assembly, etc. [8–11]. Novel image processing will be combined with data fusion from a number of sensors providing information on the sun, Earth, stars, spacecraft inertia, etc., to feed the control algorithms and provide dependable autonomous navigation. However, adjectives such as “advanced”, “complex”, and “autonomous” conceal increased processing workloads and decreased time budgets. Put into perspective, conventional space-grade processors, such as LEON3 and RAD750, can provide only a very limited level of performance (e.g., 50–400 Dhrystone million instructions per second, or DMIPS), whereas the COTS desktop central processing units (CPUs) used by algorithm designers during the early development stages of VBN techniques provide 10–100x more speed when processing images. Even with the latest space-grade devices, such as LEON4 and RAD5545 that achieve 10x more speed compared to their predecessors, or even with embedded COTS processors, the current onboard CPU performance seems to be one order of magnitude less than that necessary to execute computationally expensive vision algorithms at adequate frame rates. Therefore, to support reliable autonomous VBN, the community should consider advancing the space avionics architectures by including hardware accelerators, such as field-programmable gate arrays (FPGAs), graphics processing units (GPUs), or multicore very long instruction word (VLIW) DSP processors. That is, in short, use VLIWs or GPUs consisting of a number of cores able to execute bigger or smaller instructions and facilitate data parallelization. In contrast, the FPGAs consist of a network of much smaller processing/storage elements (such as flip-flops), which can be combined altogether to create bigger components with very deep pipelining and parallelization at multiple levels.

Designing new high-performance avionics architectures for embedded computing becomes particularly challenging when one is also required to meet stringent constraints related to space. The spacecraft resources available to electronic systems are restricted [2], with the three most critical limiting factors being mass, power, and volume. Power availability in space is low due to solar powering and power dissipation capabilities, whereas volume and mass have enormous costs per unit. Furthermore, the engineers must factor in reconfigurability, application-design complexity, fault/radiation tolerance, vibration tolerance, component pricing, etc. Last but not least, one of the most crucial factors is the achieved processing power. All of the aforementioned parameters are competing against each other in a mission-specific tradeoff manner, which becomes even more complex when the types of the aforementioned hardware accelerators are included in the architecture.

D. Paper Contribution and Outline

In this paper, we perform a tradeoff analysis for an extended number of processing units, both COTS and rad-hard. We consider image processing and computer vision algorithms for VBN (i.e., computationally intensive benchmarks), and we pay particular attention to the performance and power parameters. We define a set of representative benchmarks by studying the application scenario of VBN and collecting generic specifications and commonly used algorithms. Subsequently, we survey, test in house, and compare a wide range of microchips falling into seven main categories: space-grade CPUs, embedded CPUs, desktop CPUs, mobile GPUs, desktop GPUs, single- and multicore DSPs, and FPGAs. Notice that, besides selecting a specific device, it is of utmost importance to determine which of these architectural categories is the most promising for future mission scenarios and thus should be further supported/exploited. Put into context, our preliminary work in ESA has shown that, although multicore DSPs are gaining the attention of the space industry with 2–3 platforms currently being developed/considered, they still have insufficient processing power for high-performance applications. For example, ESA considers internally the space-oriented scalable sensor data processor (SSDP) platform based on a LEON3-FT together with Xentium DSPs [12], but this does not meet the high-performance requirements of VBN. Also, ESA considers the latest space-grade GR740 CPU, which is also not sufficient for VBN because parallelism cannot be exploited and the projected clock speed is still too low. In the past, the most frequently considered type of hardware for intensive applications were FPGAs, i.e., Xilinx Virtex-5QV and Microsemi RTG4, or the very recent European NanoXplore NX-eFPGA (dubbed BRAVE or NG-MEDIUM) [13]. Today, besides space-grade devices, cutting-edge technology includes system-on-chip (SOC) off-the-shelf devices, which constitute a very attractive solution for certain space missions, for example, ESA's e.Deorbit ADR [14].

The survey and analysis performed in the current paper will be useful for a variety of missions. It provides an in-depth exploration of a vast solution space, which can then be given as input to a focused tradeoff for the selection of avionics based on the custom-weighted criteria of a particular mission. For simplicity and to allow the consideration of many devices, we assume here generic mission specifications with demanding VBN, like in the ADR scenario: low Earth orbit, short-term life, and increased VBN autonomy. Such specifications call for increased accuracy/flexibility VBN, high-resolution images, and high frame rates, all of which contribute to increased computational workload. The assumed high-level architecture is preliminary, based on previous implementations/experience, state-of-the-art image processing, and available technologies as well as future roadmaps.

The paper is organized as follows. Section II outlines the computer vision algorithms commonly employed for VBN with an emphasis on ADR. Section III surveys the computing platforms available for use in space. Section IV performs the benchmarking and comparison of the devices. Section V highlights the most important results and concludes the paper.

II. Algorithms for Vision-Based Navigation in Orbit

Keeping track of the position of an orbiting spacecraft with respect to its surroundings is one of the key ingredients of reliable spacecraft navigation. Accurate localization information is necessary to ensure that the spacecraft follows its intended path and operates in proximity with other objects while avoiding collisions [15]. The present section reviews computer vision techniques that estimate the relative pose between a chaser and a target space resident spacecraft in the context of autonomous rendezvous, inspection, or docking scenarios involving orbiting spacecraft [16]. Despite the fact that the scope of this review focuses on the ADR scenario, it is pointed out that other applications of vision-based navigation in space rely on analogous principles and employ similar visual primitives. Such applications, for example, include obstacle avoidance, mapping, and visual odometry for planetary rovers [17,18] or approach velocity estimation and localization for precision landing [18–20]. As a result, the choice of a processing platform suitable for ADR is also highly relevant to visual navigation applications for planetary exploration rovers and landers. Figure 2 depicts some indicative applications of VBN for rovers (top left) and ADR (bottom right); an image acquired by a

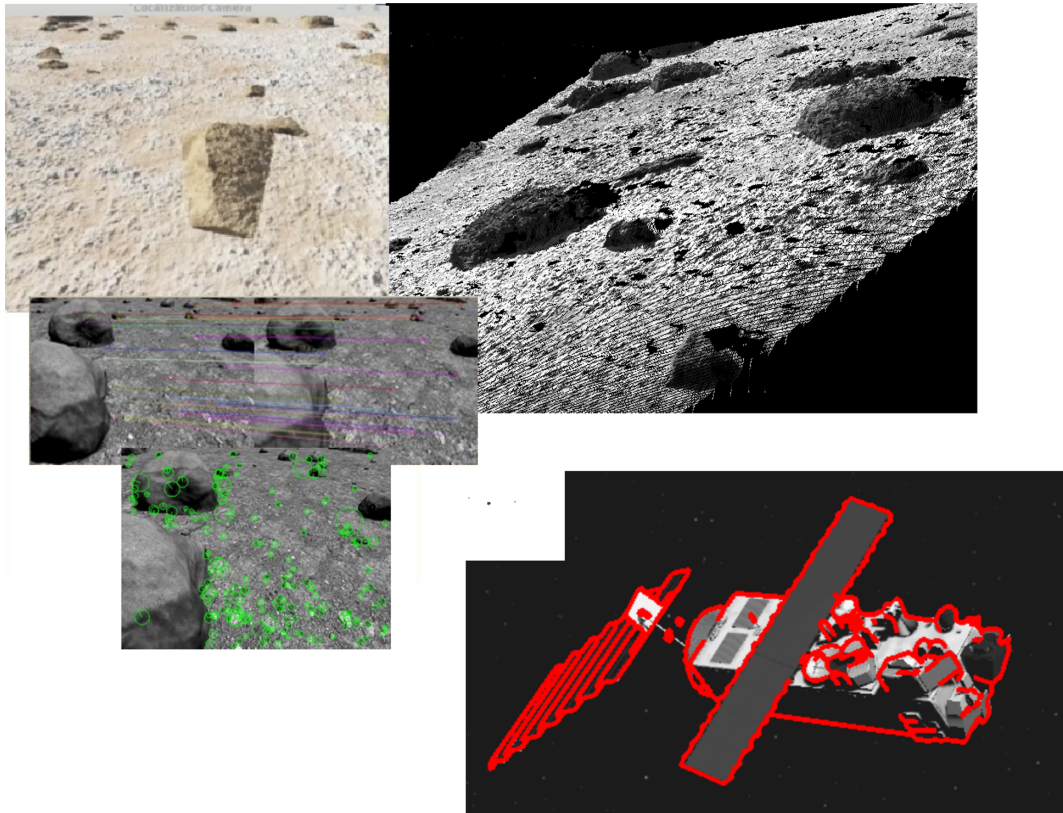


Fig. 2 Example outputs of computer vision algorithms employed in space: navigation for planetary rovers (top left) and pose for ADR targeting the ENVISAT satellite (bottom right).

Mars rover (top left) is processed by 3-D reconstruction (shown to its right, “mapping” via stereo matching) and visual odometry estimation (shown later, “localization” via feature detection and matching between successive images, annotated with circles and lines, respectively) [17], whereas edges detected on a satellite and matched against its geometric model (bottom right, red outline) serve as features for estimating the satellite’s pose relative to the camera [21].

Relative pose refers to the six-degree-of-freedom position and attitude parameters defining the geometrical relationship between the two spacecrafts. The pose estimates can be used in a closed control loop to guide the motion of the chaser spacecraft to safely approach the target or collect it as debris. Relevant approaches can be characterized by the type of sensor they employ, their operating range, and whether the target spacecraft is “cooperative” (i.e., carries markers or reflectors that are installed on it with the aim of facilitating pose estimation by the chaser spacecraft). The desired operating distance relates to the choice of sensor, sensory processing method, and type of information that can be extracted from the input [22]. The established consensus is that a single sensor will not perform adequately across the entire operating range; therefore, autonomous rendezvous and docking requires the deployment of a suite of multiple sensor systems, each with a different range and potentially principle of operation [1]. The most commonly used types of sensors are passive cameras and laser range finders.

In this review, we limit ourselves to passive camera sensors applied to pose estimation at distances below 100 m. The choice of cameras is motivated by the fact that they are attractive for use in ADR due to their compact size, low cost, simple hardware, low power consumption, and potential for dual use as navigation and visual inspection instruments. Furthermore, we focus on noncooperative targets because ADR will almost certainly concern not specially prepared or partially unknown targets.

Estimating the pose of a target in these settings is quite challenging because no strong prior information regarding its attitude and position is available. The situation is aggravated due to the varying illumination conditions, largely varying chaser–target distances that result in dramatic aspect changes, self-occlusions, background clutter, unstabilized target motion, and measurement noise. Such challenges call for elaborate and, therefore, computationally demanding processing of images. A variety of algorithmic approaches to visual pose estimation with the purpose of docking have been proposed. We note that some of these rely on prior knowledge for pose initialization (i.e., cannot handle lost-in-space configurations). These approaches as well as the majority of working systems account for the temporal continuity of a target’s motion by using tracking [23]. In this respect, by integrating past estimates, tracking can deliver a better estimate compared to that directly determined with single-frame pose estimation methods. At the same time, it can provide a prediction for the next frame, which typically accelerates pose estimation because it bounds considerably the pose search space. Vision-based pose estimation approaches can be broadly classified into two categories, namely monocular and stereo-based, depending on whether they are based on images from a single or several cameras. A comprehensive discussion is offered by Petit [24], and a brief overview follows next.

A. Monocular Approaches

Monocular approaches to pose estimation employ a single camera and typically rely on the assumption that a 3-D model of the target is known beforehand. Depending on the approach used, the 3-D model can be in the form of an unordered set of points (i.e., point cloud) and their image descriptors, a wireframe, or a polygon mesh. Use of a properly scaled model helps to resolve the well-known depth/scale ambiguity [25]. Monocular approaches infer pose by matching features detected in an image with the model. Before providing more details, a short introduction to image features is given first.

Specific structures in an image give rise to features. For example, sparse distinctive points, also called “interest points”, “keypoints”, or “corners” in the vision literature, refer to salient image locations with sufficient local variation [26]. Keypoints can be accurately localized in an

image, and their local descriptors can be robustly matched from moderately different viewpoints. Edges are another type of feature occurring when intensity changes sharply. Edges correspond to maxima of the gradient magnitude in the gradient direction [27] and are moderately robust against noise, illumination, and viewpoint changes. In general, point and edge detection is computed by using two-dimensional (2-D) convolutions with derivative filters. Popular algorithms for the detection of keypoint and edge features are those by Harris and Stephens [28] and Canny [27], respectively.

Feature-based approaches to monocular pose estimation operate by matching the detected image features with certain locations from the target's known 3-D model. Pose is then estimated so that it accounts for the observed arrangement of the detected features in an acquired image. Proposed approaches differ according to whether the image-to-model correspondences are explicitly or implicitly established. Explicit methods strive to establish (usually one-to-one) correspondences of keypoint locations, relying on their feature descriptors or their spatial arrangement [29–32]. Implicit methods attempt the establishment of correspondences through the matching of less localized features, such as edges, contours, or line segments [21,33–36]. Aiming to further increase robustness, approaches such as [37–39], which combine edge and point features, have also been proposed. We note that, despite the impressive results obtained in various application domains with state-of-the-art keypoint detectors and descriptors, their use in space is hindered by the fact that the target objects of interest lack strong texture and are often subject to illumination changes and specular reflections. Therefore, approaches based on edges are more frequently employed for ADR.

A few approaches such as [40–42] track features over time and employ simultaneous localization and mapping or structure-from-motion techniques to estimate the 3-D structure and dynamics of the target object. Pose is again computed by matching observed features to the estimated structure. Such approaches do not require any prior information about the appearance of the target and therefore are most suitable when the state of the latter is uncertain or unknown. However, their increased flexibility comes at the price of higher complexity and risk of failure.

B. Stereo Approaches

Stereo vision employs triangulation to provide estimates of the distances from the camera of the target's visible surfaces. The primary challenge to achieve this concerns the determination of correspondences among the pixels appearing in a binocular image pair [43]. Correspondence establishment involves local searches for identifying the minimum of a matching cost function that measures the affinity of pixels in the two images. For efficiency, these searches are restricted in one dimension along image scanlines, after the input images have been rectified. Commonly used stereo matching costs are the sum of squared differences (SSD), the sum of absolute differences (SAD), or the normalized cross correlation [43]. Knowledge of the baseline (i.e., the distance between the cameras), via calibration, permits scale to be determined.

To estimate pose from stereo data, either the actual surface or a set of 3-D features extracted from it are matched with a 3-D model of the target (see [44] and references therein). Stereo sensing is sensitive to the lack of texture and possible illumination artifacts, such as shadows and specular reflections, which are common in space environments. Furthermore, the camera resolution limits the depth resolution (i.e., the smallest change in the depth of a surface that is discernible by the stereo system). Depth resolution is also proportional to the square of the depth and is inversely proportional to the baseline. For these reasons, stereo measurements are more accurate at fairly short distances from a target. This is in contrast to monocular approaches presented in Sec. II.A, which are also usable at the far end of the distance range.

Indicative stereoscopic approaches to pose estimation are briefly discussed next. Stereo images are used in [45] to reconstruct 3-D data, which are then matched with an object model using the iterative closest-point algorithm to estimate pose. Sparse 3-D point clouds recovered with stereo have been employed to estimate pose, for example in [46], where the 3-D points originate from natural keypoints, and [47], which determines image points as the intersections of detected line segments. To achieve more dense reconstruction, Carlo et al. [48] suggest the combination of conventional feature-based stereo with shape from shading. It is noted that all stereo approaches still need to solve the problem of establishing correspondences between the 3-D data and the object model. Approaches that estimate pose by using stereo to detect particular structures on the target satellite have also been proposed (e.g., Xu et al. [49] extracts line segments to identify a triangular solar panel support, whereas Xu et al. [50] rely on locating a circular feature). Clearly, such approaches do not generalize to arbitrarily shaped targets.

III. Survey of Processing Platforms for Space Applications

Hitherto, the majority of space missions rely on rad-tolerant and rad-hard IBM PowerPC and Gaisler LEON CPUs. Especially in European missions, LEON is primarily used for implementing critical functions (e.g., GNC), whereas PowerPC is preferred as payload in less critical situations (e.g., onboard rovers). In terms of power consumption, the LEON-based solutions are less demanding than PowerPC (e.g., 1–2 W versus 10–20 W, respectively). In terms of radiation resilience, one can find reliable implementations in both families, e.g., withstanding 100–1000 krad (Si) [kilo rad (Silicon)] TID and suffering only 10^{-6} single-event upset (SEU) per bitday. In terms of performance, space-grade CPUs always lack 1–2 orders of magnitude from their commercial counterparts; the commercial PowerPC-750 roadmap showed a performance improvement of $\sim 2400\times$ from the 1990s to 2005 [51], whereas the rad-hard version was improved by only $\sim 300\times$. Similarly, the relatively new space-qualified dual-core Leon3-FT GR712RC provides only 140–200 DMIPS, whereas the older Intel Pentium III provides 2000 DMIPS. The latest 64-bit quad-core RAD5545 and SPARC V8 E698PM devices can deliver up to 2000–5000 DMIPS performance (i.e., they provide a tenfold increase compared to their predecessors). Table 1 summarizes space-grade processors from the two families (powerPC, LEON).

LEON evolved from ERC32 and TSC695FL SPARC V7 processor (20 DMIPS at 25 MHz) to the more advanced AT697F 32-bit LEON2-FT SPARC V8 from Atmel (43–70 DMIPS at 50–80 MHz). Developed by Saab Aerospace, the COLE SOC is based on LEON2-FT (fault-tolerant) and includes space interfaces such as MIL-1553, CAN, and SpaceWire. Airbus [52,53] uses the LEON4 high reliable processor board, providing up to 800 million instructions per second (MIPS), as well as the LEON3-based SCOC3 SOC with considerably worse performance. RamonChip [54] implemented one of the most advanced dual-core LEON3-FT devices, namely the GR712RC, which is fabricated on 180 nm complementary metal–oxide–semiconductor (CMOS) and provides up to 200 DMIPS/million floating-point operations per second (MFLOPS) at 100 MHz, dissipates less than 2 W, and withstands up to 300 krad TID, with SEU tolerance and SEL above $118 \text{ (MeV} \cdot \text{cm}^2\text{)}/\text{mg}$. Today, the top LEON-based processor is LEON4-FT (e.g., the GR740 at 65 nm), which includes a quad-core operating at 250 MHz and delivering 425 DMIPS per core; this multicore LEON4 targets up to 4300 DMIPS at 800 MHz and supports both asymmetric and symmetric multiprocessing configurations (AMP or SMP). Regarding PowerPCs developed by BAE, the RAD6000 single-board computer and the RAD750 are the most widely used. RAD750 is implemented on 150 nm rad-hard bulk CMOS and clocked at 200 MHz with 400 DMIPS and 14 W.

The aforementioned CPUs can be implemented either as softcores (e.g., in an FPGA) or as hard intellectual property on ASIC. However, it becomes clear that softcores provide considerably lower processing speed, which is around 20–70 MIPS. For example, LEON3FT-RTAX Aeroflex Gaisler on Microsemi RTAX2000s delivers 20 DMIPS, operating at 25 MHz and consuming only 0.5 W. Hence, softcores can only serve as a secondary solution for hardware/software (HW/SW) codesign of VBN. On the contrary, ARM-based ASIC CPUs integrated in new SOC FPGAs can provide more than 1000 MIPS even with single-core execution. Processors from ARM company are now being considered in multiple European space projects, for example, ARM4Space (H2020), ASCOT (CNES), as well as for the onboard computer of the Ariane 6 launch vehicle. ARM's Cortex-A9 processor is suitable for low-power applications and has been used in multiple mobile phones as well as in CubeSats.

Table 1 CPU-based processors for space with rad-hard (RH) and rad-tolerant (RT) devices

Device	Process node	CPU frequency	DMIPS	Power dissipation	Radiation tolerance
<i>PowerPC</i>					
RT RAD750 (BAE Systems)	0.25 μm radiation-tolerant	110–130 MHz	260 at 133 MHz (80 MFLOPS)	14 W at 133 MHz	TID: 200 krad (Si); SEU: $<1.6 \cdot 10^{-10}$ err/bitday; latch-up immune
RH RAD750 (BAE Systems)	0.15 μm radiation-hardened	200 MHz	400 at 200 MHz	14 W at 133 MHz	TID: 1 Mrad (Si); SEU: $<1.6 \cdot 10^{-10}$ err/bitday; latch-up immune
SCS750 SBC (3 PPC750FX) (Maxwell)	0.13 μm SOI (TRP)	400–800 MHz	1800 at 800 MHz (200 at 400 MHz)	7–25 W	TID: 100 krad (Si); SEL: >80 ($\text{MeV} \cdot \text{cm}^2$)/mg [or 50 MeV for synchronous dynamic random access memory (RAM)]
RAD5545 64-bit four-core (BAE Systems)	45 nm SOI	TBD	5200 MS (3700 MFLOPS)	17.7 W	TID: 1 Mrad (Si); SEU: $<2 \cdot 10^{-9}$ err/bitday; latch-up immune
<i>Hardcore LEON</i>					
AT697E SPARC V8 (Atmel)	0.18 μm CMOS	100 MHz	86 at 100 MHz	1 W at 100 MHz	TID: 60 krad (Si); SEU: $<10^{-5}$ err/dev/day; SEL: >70 ($\text{MeV} \cdot \text{cm}^2$)/mg
TSC695F SPARC V7 (Atmel)	0.50 μm SOI	10–25 MHz	20 at 25 MHz	1 W at 25 MHz	TID: 300 krad (Si); SEU: $<3 \cdot 10^{-8}$ err/dev/day; SEL: >70 ($\text{MeV} \cdot \text{cm}^2$)/mg
GR712RC dual-core SOC (RamonChip)	180 nm CMOS	100 MHz	140–200 DMIPS (200 MFLOPS)	1.5 Wx2	TID: 300 krad (Si); SEL: >118 ($\text{MeV} \cdot \text{cm}^2$)/mg; SEU tolerant
<i>Softcore LEON</i>					
GR-PCI-XC5 V SPARC V8	65 nm	50 MHz	70 DMIPS (50 MFLOPS)	TBD	RH by design (Virtex5QV tolerance)
LEON3FT-RT3PE on RT ProASIC3 (Gaisler, Actel)	130 nm	20–25 MHz	20 DMIPS	0.2 W	TID: 15–25 krad (Si); SEU: >6 ($\text{MeV} \cdot \text{cm}^2$)/mg; SEL: >96 ($\text{MeV} \cdot \text{cm}^2$)/mg
<i>Higher-performance and multicores</i>					
P4080 Board		1.5 GHz	27.6 DGIPS	45 W	Latch-up immune; Virtex-5 voting system
Intel Atom TMR board		1.6 GHz	3.3 GIPS	25 W	Radiation-tolerant
High reliable processor board (Airbus, SPARC V8 LEON4, Virtex-5 I/O, PikeOS)		150 MHz	800 MIPS	TBD	TBD
GR740 (four-core LEON4FT+2 FPU)		250 MHz	425 DMIPS	1.8 W	Latch-up immune (RH-bd, on ST's 65 nm)
OCE E698PM (RH, four-core 32-bit SPARC v8, MAC/UMAC DSP 64-bit double precision FPU)		600 MHz	2100 MIPS (900 MFLOPS)	2.8 W	TID: >100 krad (Si); SEL: >120 ($\text{MeV} \cdot \text{cm}^2$)/mg; SEU: >37 ($\text{MeV} \cdot \text{cm}^2$)/mg

NASA [55] has exposed the Snapdragon APQ8064 IFC6410 board to Ar at linear energy transfer (LET) = 7 ($\text{MeV} \cdot \text{cm}^2$)/mg. However, up to now, only the ARM Cortex-M0 has been implemented as radiation-tolerant by triplication, to provide no more than 50 DMIPS. In more-powerful CPU-based scenarios, the P4080 is based on octo-core PowerPC plus Xilinx Virtex-5 FPGA for voting to provide up to 27.6 DGIPS (Dhrystone giga instructions per second), however, at the cost of more than 25 W power dissipation. At similar power budget, the Intel Atom TMR board provides up to 3.3 GIPS (giga instructions per second) with radiation tolerance being provided through triplication. Finally, we mention that ESA's GAIA mission uses a video processing unit with a SCS750 board (commercial PPC750FX) providing 1800 DMIPS at 800 MHz, but consuming 33 W and requiring mitigation via triple redundancy of the PPC plus voting with Actel FPGA to withstand radiation.

Beyond the CPU-based approach, the most common general-purpose accelerators are FPGAs, VLIW DSP multicore processors, and GPUs, which, however, are not widely considered for space applications due to their increased power dissipation. The VLIW DSP solution is being examined through European projects such as MACSPACE (with 64×1643 DSP cores from CEVA) and MPPB/SSDP [56] (with Xentium VLIW DSP cores from Recore), or devices such as the eight-core C66x DSP Texas Instruments (TI) 66AK2H12, as summarized in Table 2. The conventional single-core VLIW space-grade DSP devices (e.g., TI SMV320C6727B) deliver inadequate processing power to be considered for high-performance applications. Instead, TI 66AK2H12 offers two orders of magnitude more processing power, however, without space qualification yet. Similarly, at only 10 W, the European MACSPACE 64-core DSP will also provide one order of magnitude higher performance than single-DSP solutions.

Regarding space-grade FPGAs (Table 3), given the increased requirements of image processing in terms of logic and memory, the most suitable devices for VBN seem to be the Xilinx Virtex-5QV and the Microsemi RTG4. Between these two, the most preferable is Virtex-5QV due to its double on-chip memory, which is a critical resource for image processing. The space-grade European market offers today the ATMEL devices, which are considered quite small for high-performance tasks. However, the recently introduced European NanoXplore NX-eFPGA should be able to support high-performance tasks in the near future. Also in the near future, it is expected that the Zynq Ultrascale+ SOC FPGA will qualify as rad-tolerant and provide more FPGA resources than any other space-grade device (e.g., 10x more logic and 7x more memory compared to Virtex-5QV). Currently, multiple research groups consider avionics architectures using FPGAs, even with COTS devices. More specifically, when combining trends such as high-performance, SOC, and COTS, the most reasonable solution studied is the Xilinx Zynq FPGA [57,58]. The Jet

Table 2 DSP-based processors (space-grade and COTS)

Device	Process node, nm	CPU frequency	Power dissipation, W	DMIPS, GFLOPS	Radiation tolerance
MPPB/SSDP (TAS-I and Recore, LEON2 + 2/4 Xentium DSPs)	90	80 MHz	1.5 (min)	TBD	TBD
TI SMV320C6727B (C67x + single-core VLIW DSP)	130	250 MHz	TBC	1.5	TID 100 krad (Si) and SEL immune up to LET = 117 ($\text{MeV} \cdot \text{cm}^2$)/mg
TI 66AK2H12 SOC (four-core ARM Cortex-A15 + eight-core C66x DSP)	28	1.4 GHz	10	160	Commercial device, not characterized for radiations
MACSPACE RC64 Manycore (GR712RC dual-core LEON3 + 64x CEVA X1643 DSP cores)	65	300 MHz	10	38	Targeting space (300 krad TID and SEL immune)

Table 3 FPGA-based processors (space-grade and COTS)

FPGA	Logic	Memory	DSPs	Power	Technology	Radiation tolerance
Xilinx Virtex-5QV	81,920 LUT6	596 RAMB16	320	5–10 W	65 nm SRAM	SEE immune up to LET 100 MeV/(mg · cm ²) and 1 Mrad (Si) TID
Microsemi RTG4	151,824 LE	5.3 Mbit	462	1–4 W	65 nm flash	SEE immune up to LET 110 MeV/(mg · cm ²) and TID > 100 krad
Microsemi ProASIC3	35,000 LE	0.5 Mbit	—	~2 W	130 nm flash	40 krad TID
Microsemi RTAX	4 million gates (~37,000 LUT)	0.5 Mbit	120	TBD	150 nm antifuse	SEL immune up to 117 MeV/(mg · cm ²) and 300 krad TID
ATMEL ATFEE560	560,000 gates (28,800 LUT4)	0.23 Mbit	—	TBD	180 nm SRAM	SEL immune up to 95 MeV/(mg · cm ²) and 60 krad TID
NanoXplore NG-MEDIUM	34,272 LUT4	56 RAMB48	112	TBD	65 nm SRAM	SEL immune at 60 MeV/(mg · cm ²) and 300 krad TID
Xilinx Zynq7000 SOC	277,400 LUT6	1510 RAMB18	2020	3–6 W	28 nm SRAM	Not space-qualified
Microsemi SmartFusion2 SOC	146,124 LE	4.5 Mbit	240	~2 W	65 nm flash	Not space-qualified
Xilinx Zynq Ultrascale+ SoC	523,000 LUT6	70.6 Mbit	1968	TBD	16 nm SRAM	Not space-qualified

Propulsion Laboratory proposes for NASA the APEX [57], an FPGA-based SOC, which is being prototyped on Zynq-7Z100. Next to the Zynq chip, APEX integrates unvolatile solid-state and double-data-rate memories, all on ZedBoard mini-ITX, plus an FPGA mezzanine card (FMC) connected card providing analog to digital converter functionality. Additionally, Iturbe et al. [57] propose to combine certain functionality of the COTS Zynq such as internal watchdogs with mitigation techniques on the FPGA side to increase the resilience of the system to space radiation. Overall, the volume of this avionics module is $17 \times 17 \times 5 \text{ cm}^3$, whereas its power dissipation is around 5 W. The CHREC space processor [58] is also based on Xilinx Zynq7000. The developed board includes rad-hard flash memory and has a form factor of 1U. This means that it fits in a $10 \times 10 \times 11.35 \text{ cm}^3$ CubeSat, weighing less than 0.1 kg and consuming around 2 W of electrical power. NASA Goddard Space Flight Center proposes using SpaceCube [59,60], which is an architecture based on space-grade FPGAs (e.g., Virtex-5QV). They develop a hybrid computing platform that consists of a CPU, DSP processors, and FPGA logic, with various versions already tested in a space environment. “SpaceCube mini” is the smallest module, with a form-factor 1U consisting of three U-folded boards, one of which includes the Xilinx Virtex-5QV as the main processing unit, weighing less than 1.4 kg (with a 2.5-mm-thick housing) and consuming around 5 W, whereas its estimated TDI is 50–700 krad (Si). “Spacecube 2.0” is larger, including four boards mounted in a rack of 3U form factor and having a volume of $12.7 \times 17.8 \times 22.9 \text{ cm}^3$, a mass of 5.8 kg, and consumption of 20 W. Finally, EREMS has developed the ELISE main onboard computer based on Zynq-7030, which dissipates 5 W, is SET/SEU/SEL protected, and withstands ~5 krad.

Following the preceding survey, we summarize a number of high-level choices for designing a high-performance architecture with: 1) many-core CPU, 2) CPU plus multicore DSP, 3) CPU plus FPGA, 4) multi-FPGA including softcore CPU, and 5) system-on-chip. Choices 2–4 assume distinct devices interconnected with (e.g., a high-speed network like SpaceWire at 100 Mbps). The drawback of such an approach would be the increased size, mass, and power dissipation due to the multiple chips involved. Moreover, coprocessing bottlenecks could arise in cases 2–4 due to congestion or slow interconnects among the devices, which would result in limited communication bandwidth between intermediate steps of an algorithm. When considering multicores running SW, delivering a hard real-time system implies functional isolation together with time analysis against single-core chips. This is difficult due to state interruption between processes/applications, constraints of high-criticality over low-criticality applications, as well as safety and worst-case execution time requirements of SW, as is the common practice in the space industry. Instead, FPGAs can avoid such complicated SW analysis, although they require increased development time. Softcore CPUs (e.g., LEON synthesized in an FPGA) provide one order of magnitude lower performance than hardcore IPs integrated in a SOC device (e.g., ARM Cortex A9 in Zynq), which can execute more demanding functions in a HW/SW codesign scenario. By taking into account the preceding factors, we identify the SOC approach as the most promising for high-performance embedded computing. SOC devices offer a wide range of advantages owing to the coupling of acceleration resources with general purpose processors and peripherals. They can lead to architectures with single-device, low-power, small size/weight, and rapid intrachip communication able to support efficient coprocessing.

IV. Comparison and Evaluation of Processing Platforms

This section compares candidate platforms, primarily, in terms of throughput, power, and performance per watt, and secondarily, in terms of connectivity, size/mass, and error mitigation opportunities. We evaluate their capabilities in relation to image processing and computer vision tasks, with the intention of selecting the most suitable platform for the future space missions under consideration. Toward a fair/accurate comparison, we combine various literature results and in-house development/testing, and we use a number of benchmarks common to all platforms. Table 4 summarizes the devices and algorithms used in our study. Specifically, we examine 30 representatives from all four processor

Table 4 Summary of platforms and benchmarks considered in our comparative study

Platform	In-house development and testing	Literature survey
FPGA	Xilinx Virtex6 VLX240T-2, Zynq7000 (Z7020, Z7045)	Altera Stratix III E260, Xilinx Virtex 5QV
CPU	desktop Intel i5-4590, laptop i3-4010U embedded: ARM CortexA9, Intel X1000 space: LEON3, OCE E698PM (LEON4)	Virtex4 VLX100, Virtex6 V5X475T, Zynq7000 Desktop: Intel i7-3820, AMD FX-8120 embedded ARM CortexA15
GPU	Nvidia GeForce GTX 670, GTX 680, GTX 960	space-grade BAE RAD5545 Nvidia GTX 980Ti/650Ti/295, Tesla C2050, Mobile: Nvidia Tegra K1/X1, ARM Mali-T760
DSP	Space-grade Xentium MPPB embedded multicore Myriad2	TI multicore TMS320C6678 and 66AK2H14, single-core C674x, 64-core RH RC64 (MACSPACE)
Benchmark	2-D convolutions (5×5 to 11×11), Harris corner and Canny edge detectors, stereo matching, hyperspectral search, pose estimation (including feature detection, description, matching), superresolution	2-D convolutions and SAD (up to 25×25), Harris corner and Canny edge detectors, stereo matching, image denoising and Block matching, hyperspectral imaging, etc. (plus nominal DMIPS and MFLOPS figures)

categories (namely CPU, FPGA, GPU, and DSP) by using more than 10 well-known benchmarks applied to images of size 512×384 and 1024×1024 pixels. These benchmarks were selected as a representative set of the operations required by algorithms surveyed in Sec. II.

The comparison of such diverse platforms becomes very challenging due to the dependence of the relative performance of each benchmark on every device on such factors as 1) the distinctiveness/peculiarities of each computational and programming model; 2) the programming effort, coder's experience, and code optimization level of each benchmark; 3) the application/algebraic domain targeted by each architecture; and 4) the technology node and size of each chip. To tackle these issues, along with the plethora of chip–benchmark combinations that emerge, we follow a two-stage pruning method tailored to our goals. First, we perform a general “CPU versus GPU versus DSP versus FPGA” evaluation, and then we proceed to a more in-depth comparison of the 4–5 most promising chips. To address item 4 in the second stage, we focus on 28 nm technology node and, in particular, SOC devices. To address item 3 in both stages, we focus on image processing. To address item 2, we select published works that devote equal effort to their examined platforms, avoiding comparisons of optimized Compute Unified Device Architecture (CUDA) versus generic high-level-synthesis implementations, and we assign distinct, experienced developers to each platform tested in-house. To address item 1, we rely on the multitude of results and their convergence to consistent conclusions by experienced developers. Finally, we note that our primary goal is to derive the relative performance of the devices rather than absolute numbers for specific algorithms/datasets. Thus, we avoid running the exact same test on all devices, which would become impractical for 10 benchmarks, 2 image sizes, and 30 chips. Instead, we proceed by extracting multiple individual performance ratios for key pairs of devices, and then, as explained in Sec. IV.C, we combine all ratios/comparisons in a unified table.

For clarity of presentation, Secs. IV.A and IV.B report separately the most important results derived from the literature and the measurements performed in-house for both stages of the aforementioned comparison strategy. These sections focus on performance and power consumption, while combining absolute numbers and extrapolations/estimations to facilitate straightforward comparisons between the devices. Subsequently, Sec. IV.C merges all results to provide a unified perspective. Additionally, Sec. IV.C considers factors like size/mass of the COTS board, development time, and COTS cost and discusses suitability to space missions.

A. Literature Results

The literature includes a multitude of implementations on various devices, which we use here to construct our first set of performance comparisons. Primarily, we collect results regarding platforms not available in-house, and secondarily, we include papers to enrich and verify our own tests.

Regarding mobile GPUs, Nah et al. [61] perform a wide platform comparison that, among others, includes Nvidia Tegra and ARM Mali devices. They employ a set of nine computer graphics benchmarks (L-Bench) with various workloads. On average, their results indicate that Tegra K1 (365 billion floating-point operations per second, or GFLOPS) is performing 1.89x faster than ARM Mali-T760 MP6 (143 GFLOPS). However, the Mali-T760 MP6 consumes ~ 6 W, whereas K1 consumes ~ 10 W, and hence, these mobile GPUs provide similar performance per watt. We note here that the Mali GPU is integrated in the latest multiprocessor system-on-chip (MPSOC) FPGAs of Xilinx; the MPSOC uses Mali-400, providing only 21 GFLOPS and performing one order of magnitude slower than K1. The latest Tegra X1 provides 1.4x higher throughput compared to Tegra K1 (1–1.8x according to various benchmarks), and hence, we can infer that these high-end mobile GPUs span a performance range of 1–3x. Gao et al. [62] compare the mobile Tegra K1, to desktop GPU Nvidia GTX 680, to the many-core DSP TI 6638K2K, and to Intel Core i7-3770K CPU, by employing a set of sparse matrix-vector multiplication benchmarks. On average, they show that K1 is 9x slower than GTX 680; however, it achieves almost similar performance with the single-threaded desktop CPU; they use Intel's Math Kernel Library (MKL) library for parallelization on CPU, but we use here single-thread extrapolations to facilitate our own comparisons. Han et al. [63] implement a compression method for deep neural networks and evaluate the performance of their pruned network on Intel i7-5930K CPU, Nvidia GeForce GTX Titan X, and Tegra K1, also employing optimized libraries and MKL, from which we make rough estimations for single-threaded execution. They show that Tegra K1 is ~ 3 x faster than the single-threaded CPU and one order of magnitude slower than Titan X, however, with nominal 250 W. Similarly, for telecom benchmarks including data transfer, Kim and Bhattacharyya [64] show that mobile GPUs are up to 10x slower than desktop GPUs (Tegra K1 versus Nvidia GTX 680/780ti/970).

Regarding embedded CPUs, Mitra et al. [65] consider the performance gain obtained via single-instruction, multiple-data processing and the comparison of conventional CPUs to embedded ARM processors, such as Cortex-A9 (integrated in SOC devices, e.g., OMAP 4460, Exynos 4412, Tegra T30). Their results indicate that, for the single-threaded benchmark of simple binary thresholding on images, the Core i5-3360M CPU achieves 20–24x faster execution compared to Cortex-A9 (reduced at 667 MHz), whereas for a more complex benchmark such as edge detection, the i5 CPU is 21–32x faster than the embedded Cortex-A9. Peng et al. [66] use N-body simulation as a benchmark and show that, for single-threaded execution, the Intel i5-2467M is 12.8x faster than the embedded Cortex-A9 (at 667 MHz). Furthermore, they employ optimized CUDA coding on GPUs to show that the mobile Tegra K1 is 61.8x faster than Cortex-A9, whereas high-end desktop GPUs achieve a huge gain over A9, in the area of 1000x (e.g., 2032x versus Nvidia Tesla K80 with nominal 300 W). We note that, in a broader sense [66,67], the throughput of embedded CPUs could increase even by 10x when we consider higher clock rates (e.g., 2.3 GHz) and/or bigger microarchitectures (e.g., ARM Cortex A-57). However, for the sake of categorization, we focus here on more conservative scenarios, such as Cortex A9/A15 operating below 1 GHz.

Regarding high-end many-core DSPs, such as TI's TMS320C6678 and 66AK2H, which both use the C66x core, Sioutas et al. [68] and Kumar et al. [69] compare the eight-core C66x to ARM Cortex A15 and get ~ 9 x faster execution for various benchmarks (4–10x). By reducing MIPS and MHz, as well as by considering the results of [66], we derive that the A15 processor at 1.2 GHz is 3–5x faster than the Cortex A9 of Zynq (at 667 MHz), which we use hereafter as reference. Hence, we deduce that 66AK2H14 is ~ 36 x faster than A9. For hyperspectral unmixing, when comparing the eight-core C6678 at 1 GHz to ARM cortex A9 at 1 GHz, Castillo et al. [70] derive slightly smaller factors than the preceding averaged ~ 36 x. If we adjust their ARM A9 to 667 MHz and their eight-core C66x to 1.2 GHz as in 66AK2H, we conclude that the DSP is up to ~ 25 x faster than the single-core ARM CPU. Menant et al. [71] use a stereo matching benchmark, which in terms of complexity is roughly similar to our own “disparity” that is detailed in [72], on a single-core C66x to achieve 14 frames per second (FPS) for small images. They also show in [73] that the speedup to eight-cores is ~ 4 x, and hence, they would achieve ~ 56 FPS on 66AK2H14. By making extrapolations from our own Vrtex6 disparity to emulate their complexity (we assume 434×380 resolution images with 18 candidate disparities per pixel, and by computing only one map per image, thus, we decrease our workload by 11x) and by employing four-engine parallelization on Zynq (to consume 1/4 cycles at 350 MHz instead of 283 MHz clock, due to the technology scaling), we estimate a throughput of ~ 337 FPS on FPGA. Thus, we get a coarse comparison showing that Zynq is ~ 6 x faster than 66AK2H14 for disparity. For the Harris corner detection benchmark, considering cvCornerHarris on the C674x core [74], we can roughly estimate that 66AK2H14 will complete a 1 megapixel image in ~ 50 ms, and hence, it is ~ 4 x slower than our Zynq implementation (see Sec. IV.B.2). Ramesh et al. [75] test the eight-core TMS320C6678 and show that, for frequency-domain finite impulse response, it achieves 1/3 throughput versus Nvidia Tesla C2050, whereas for 25×25 convolution kernel, the TMS320C6678 achieves similar throughput with Nvidia GTX 295, or 1/3 throughput compared to FPGA Stratix III E260, or 1/9 compared to our own Zynq-7000 implementations reported in Sec. IV.B.2 (90 versus 10 ms for 1080p images). It is also shown in [75] that code optimization is

important because it boosts the DSP performance by 2–3x. Finally, based on the results of [62] for matrix multiplication benchmarks, we conclude that 66AK2H14 achieves similar throughput with Tegra K1 GPU and single-threaded Intel Core i7-3770K. Regarding the 64-core DSP computer RC64 “MACSPACE” [76], in lieu of relative benchmarking, we consider the nominal values: 10 W for 20 single-precision GFLOPS (75GMACS/150GOPS) when operating at 300 MHz. This FLOPS value is half that of 66AK2H14 (GMACS are similar) when also operating at 300 MHz. Hence, for mixed fixed/floating-point arithmetic benchmarks, we expect that RC64 will achieve at most similar throughput with 66AK2H14.

Regarding direct comparisons of GPU and FPGAs, Tippetts et al. [77] perform an extensive literature survey for such platforms for stereo matching benchmarks. They examine dozens of algorithms and implementations, but their results concern devices that are now 5–10 years old (e.g., Intel Core2duo, Nvidia GTX 280, and TI C64x+). When looking at the cloud of results collected in their study, as a general conclusion, one can remark that GPUs and FPGAs are difficult to distinguish with respect to throughput; using the paper’s metrics for the fastest 50% of the implementations in each category, GPUs achieve 0.1–7 Gde/s, whereas FPGAs achieve 1–6 Gde/s. This picture is consistent with our own findings. Similarly, for older devices and various image processing benchmarks such as stereo matching, face detection, object detection, and beamforming, some authors like Chen et al. [78] show $\pm 10\%$ throughput differences when comparing specific devices, whereas others show differences around 2–4x, either favoring GPUs [79] or FPGAs [80]. The survey in [77] also includes DSP processors, which however are 1–2 orders of magnitude slower than GPUs and FPGAs. This gap seems to be narrowing due to the latest high-end many-core DSPs considered in the current paper (see Sec. IV.C). Considering vision algorithms for space [81], the desktop GPU achieved one order of magnitude higher performance per watt than the desktop CPU (i.e., up to 12x with respect to GFLOPS and 49x with respect to GFLOPS/W). Finally, we note that, for GPU–GPU comparisons (e.g., to combine results among papers and derive indirect comparisons), or to extend/support our own conclusions, we occasionally consider the variety of GPU benchmarks available online.

B. Development and In-House Testing

1. Central Processing Unit

Our in-house development and testing involves a number of CPUs ranging from small space-grade and embedded devices to big desktop/laptop platforms. The C/C++ benchmarks used are Harris corner detection (employing five 2-D convolutions [17,28]), hyperspectral searching (of prestored spectral signatures in a pixel-by-pixel fashion [82] by using metrics such as χ^2 , L_1 , L_2 , cf. [83]), Canny edge detection [27], and binocular visual odometry [17] (including distinct algorithms for feature detection, feature description, temporal and spatial matching, pose estimation [84], outlier removal, etc.). The code used is fairly optimized and common among devices, with a reasonable level of compiler optimization (e.g., gcc version 4.6.3 with the -O3 flag), and the execution is single-threaded in all cases, so that more straightforward comparisons can be made.

Harris was implemented on embedded Intel Quark X1000 using a Galileo board [85], embedded ARM Cortex A9 on Zynq7020 at 667 MHz [86], on desktop Intel Core i5-4590, i3-4010U, and i7-3820, as well as on space-grade LEON3 implemented as software on FPGA at 50 MHz and LEON4 on chip OCE E798PMPM at 200 MHz. The execution time for a 512×384 image was 1054–2177 ms on X1000 (depending on the use of yocto linux 3.8.7 OS or bare metal), 456 ms on ARM A9 with petalinux, and 10–20 ms on the various Intel Core devices with ubuntu linux. On the space-grade devices with RTEMS OS, we obtained 5 s on LEON3 and 0.55 s on LEON4. This common benchmark provides an accurate relative performance evaluation of the platforms showing that space-grade CPUs are slower than embedded ones, which in turn are 25–50x slower than desktop CPUs. This 25x factor is also derived, on average, when using our hyperspectral benchmark to compare ARM A9 to the small desktop CPUs (i.e., Intel i3-4010U and i3-3110M on laptops); depending on the benchmark configuration, the execution time for 1 megapixel image ranges from 0.4–62 s on i3 to 3.5–1702 s on A9. When testing Canny on 1 megapixel grayscale images, the relative performance of desktop CPUs becomes smaller than the aforementioned, i.e., only 10x versus ARM and 40x versus X1000 (51 ms on i5, 502 ms on A9, 2120 ms on X1000).

For the space-grade CPUs, to evaluate the performance gain of the latest LEON generation, Table 5 analyzes the execution time of our visual odometry benchmark [17] processing stereo pairs of 512×384 pixels. We remark that the first three columns/functions are executed twice, and the entire pipeline is completed in 0.25 s on Intel i5-4590. On average, with a limited deviation among functions, the LEON4 achieves a 9.2x gain due to its 4x higher clock rate and improved architecture. If we assume further clock increase to 600 MHz and very effective parallelism to four cores, then we could anticipate future gains of even up to 100x. Even so, we note that such impressive performance improvement will not be sufficient for high-definition image processing at high frame rates. For example, with the given algorithm, we would process only ~ 1 FPS of size 512×384 , which is one order of magnitude less than, for example, 1 megapixel at 5 FPS. That is, the space-grade CPUs are still 1–2 orders of magnitude slower than necessary for computationally demanding vision-based navigation, and hence, some form of acceleration is essential.

2. Field-Programmable Gate Array

To perform in-house evaluation of high-performance FPGA platforms, we accelerate multiple benchmarks on Xilinx Virtex6 and Zynq-7000 devices with hand-coded very high speed integrated circuits (VHSIC) hardware description language (VHDL). The majority of these accelerators were originally developed and carefully optimized within completed ESA projects targeting vision-based navigation for planetary rovers [87]. First, we focus on the Harris benchmark, which includes diverse operations, such as 2-D convolutions, evaluation of floating-point formulas, and repetitive comparisons. Our design is based on sliding windows, deep pipelining at pixel-basis, and high parallelism of arithmetic operations, such that we can process all intermediate values/pixels with a rate of one value per cycle. The VHDL design is an optimization/reorganization of the architecture detailed in [17]. Here, we consider 1024×1024 8-bit images, and we deploy four parallel Harris engines, one per image quadrant. Moreover, to account for the usual HW/SW codesign and I/O scenarios of SOC devices, we assume that the accelerators in the programmable logic (PL) are fed by the processing system (PS), and we develop a PS-PL communication scheme based on AXI and Xillibus [88]. Depending on burst size, the achieved bandwidth is 1.3–2.6 Gbps, and hence, a 1 megapixel image is transferred in around 3.2 ms. The total resources used by Harris on Zynq-7045 PL are 30,612 LUTs (12%), 43,100 DFFs (delay flip flop) (10%), 186 RAMB36 (34%), and 216 DSPs (24%). By operating at 200 MHz, the processing time is 8.8 ms, and the power is 4 W, distributed as 0.6 W per Harris engine and 1.6 W for the ARM processor. Therefore, Zynq-7045 is 233x faster than Cortex A9 (2.8 s per image) and 5–9x faster than desktop CPUs.

Table 5 Profiling of a complete visual odometry (pose estimation) pipeline on space-grade LEON3 and latest LEON4 (E698PM) CPUs; pipeline stage descriptions are provided in [17]

	Harris	ANMS	SIFT	Match-T	Match-S	Other	Total
LEON3 (single-core, 50 MHz)	5, s	15.3, s	29.7, s	20, s	4.8, s	<5, s	129, s
LEON4 (single-core, 200 MHz)	0.55, s	1.7, s	2.7, s	2.3, s	0.32, s	<1, s	14, s
Gain	9x	9x	11x	8.7x	15x	—	9.2x

Furthermore, we also test stereo matching, which is a very intensive benchmark involving highly repetitive calculations. Specifically, our stereo matcher concerns bidirectional disparity map generation with 7×7 Gauss-weighted cost aggregation, and its architecture is detailed in [72]. For one pair of 1120×1120 8-bit pixel images with 200 disparity levels, a four-engine parallelization on Virtex6 VLX240T requires 0.54 s. Therefore, the FPGA is 29x faster than Intel i5 (see [89] for detailed comparison) and approximately 7700x faster than space-grade LEON3; this factor was derived after extrapolation from smaller images due to memory limitations on LEON3. Such huge factors can be also derived for Harris detection and scale-invariant feature transform (SIFT) description. For the former, comparing the aforementioned 12 ms total time of Zynq-7045 to the almost 27 s of LEON3, which is an extrapolation based on image size and Table 5, leads to a factor of ~ 2250 x. Even when we estimate the performance on a space-grade FPGA like Virtex-5QV (52% LUT (look-up table) and 55% RAM utilization, with 14.7 ms processing time due to 120 MHz clock), the total speedup is still ~ 1500 x. For the latter, we optimized our SIFT descriptor architecture, which is presented in [17], to use fixed-point arithmetic and smaller description window (33×33), and hence, we improved the execution time by 30% on Virtex6 to consume 28 ms when processing 1200 features. When porting to Zynq-7045 (200 MHz) and assuming four-engine parallelization, even if we add the 3.2 ms overhead of PS-PL communication, we obtain less than 10 ms total execution time (i.e., an estimated ~ 3000 x speedup versus LEON3, versus SIFT in Table 5).

To test more streaming applications relying on continuous pixel I/O, we assume a faster PS-PL communication scheme than the one implemented previously using Xillibus [88]. We assume the four AXI VDMA scheme developed by Xilinx [90], which achieves 12 Gbps transmission rate and another 12 Gbps reception. In our hyperspectral benchmark that involves matching of incoming images to known spectral signatures, we develop in VHDL a large systolic network of processing elements to parallelize the pixel subtraction and metric accumulation, e.g., for L_1 , $\sum_i |x_i - y_i|$ as well as the comparison to distinct pixel signatures. When we assume ordinary application scenarios and up to 12 Gbps PS-PL communication, the Zynq-7045 PL operates up to 428 MHz and accelerates its own ARM Cortex A9 by 340–630x. For more complex scenarios (e.g., more elaborate metrics like χ^2 , hundreds of examined signatures, or 100 Gbps bandwidth), the FPGA acceleration would approach 10^4 x; however, in the current paper, we focus on the more general case for fair comparisons.

3. Graphics Processing Unit

Based on the same C/C++ benchmark executed on the CPUs, we implemented Harris in CUDA and tested it on our desktop GPU Nvidia GTX 680 (Kepler microarchitecture at 28 nm, 1536 cores at 1 GHz). Our implementation uploads a 512×384 image to the GPU and sequentially executes the steps of Harris by employing parallelism on a pixel basis (i.e., one pixel per thread). Harris involves separable convolutions for image derivatives $\times 2$, squared derivatives computation $\times 3$, Gauss smoothing $\times 2$, and cornerness calculation. For all kernels, we use the GPU cache for reading the pixels and the constant memory of the GPU to store the convolution kernels. The GPU achieves a throughput of around 1000 FPS (i.e., 1 ms per image), with 1/3 of the time spent on processing (e.g., 0.052 ms for a single 2-D convolution with a 7×7 separable kernel, or 0.033 ms for a 5×5 kernel) and the other 2/3 dedicated to communication. Our four-engine FPGA Harris with 200 MHz clock would spend for the same image 1.6 ms on PS and 0.13 ms on PS-PL communication (assuming 12 Gbps bandwidth); therefore, the GPU is faster than Zynq by a factor of 1.8x. In terms of performance per watt, the Zynq FPGA outperforms the GPU by a factor of more than 10x. The GTX680 is 10–20x faster than desktop CPUs and up to 500x faster than ARM A9. Compared to high-end mobile GPUs, we expect/estimate the GTX680 to be 6–10x faster than Tegra X1 due to 7x bigger memory bandwidth and 6x more CUDA cores.

Besides Harris, we also use image superresolution (SR) and stereo matching (disparity [72]) to compare desktop GPUs to FPGAs. We tested a variety of devices and performed in-depth optimization of the implemented benchmarks on both platforms; a detailed analysis has been published separately [89]. Here, we consider Nvidia GTX 670/960, whereas for the FPGA (Zynq7000 Artix), we consider both processing and communication time, in contrast to the study in [89] that focuses on processing. The assumed 12 Gbps PS-PL bandwidth on Zynq can support continuous operation of only up to three SR processing engines at 120 MHz or 2 engines operating at 180 MHz. Hence, a 1920×1080 image can be upsampled by Zynq in 5.7 ms. Hypothetically, the FPGA performance could increase by up to 10x if we assume a larger chip and a more efficient means of communication feeding the fabric with the very high I/O of SR. The GPUs can process the same image in 2.9–9 ms with 67–76 W measured power (i.e., they are up to 2x faster than FPGA). However, for smaller images, because of limited occupancy, GPUs can become 10x slower. For the stereo matching benchmark, as mentioned in Sec. IV.B.2, our FPGA requires 0.54 s, and our GPU implementations on GTX 670/960 require 0.5–0.36 s. Notice that, in this benchmark, the I/O is not a bottleneck, and processing could be supported even with 1 Gbps communication. Moreover, on Zynq-7045, because of the technology scaling (versus Virtex6), we can expect/estimate the execution time to decrease to 0.4–0.5 s and provide almost equal throughput with the high-end GPU.

4. Digital Signal Processor

To perform in-house evaluation of high-end many-core DSP-oriented platforms, we implemented Harris corner detection on the 12-core VLIW Movidius Myriad2 [91]. By applying a number of source-code modifications to exploit the underlying architecture, we efficiently used the limited local memory and exploited the long instructions of Myriad2. In particular, we first identified the most frequently accessed data structures and placed them in the local memory. Second, we identified and removed storage redundancies. Third, we partitioned the image in 12 slices and assigned them to distinct cores. Fourth, we applied platform-oriented modifications, such as employing one 2-D convolution kernel instead of two one-dimensional separable filters, aiming to improve HW utilization and decrease memory accesses. Steps 1 and 3 yielded the highest performance improvements (i.e., 35x gain with 25x less energy with respect to the initial porting). The final result shows that, for 1 megapixel image, execution on 12 VLIWs at 600 MHz requires 110 ms and 1 W. Therefore, compared to the aforementioned four-engine Zynq7045 implementation, Myriad2 is 9x slower, but only 2x worse in terms of performance per watt due to its very low power consumption. We note that, when compared to Zynq-7020 processing 512×384 images with only one Harris engine (i.e., when compared against smaller devices with limited room for parallelization and smaller input), Myriad2 achieves similar performance per watt and is only 2.5x slower than the FPGA (execution times are 8 ms/2 W on Zynq-7020 versus 20 ms on Myriad2, which approaches the Intel i5 performance).

Besides accelerating Harris on 12 VLIWs, we evaluated Myriad2's integrated ASIC accelerator. We note that the computational complexity in this version of Harris is lower than our own Harris benchmark, which also includes convolutions for Gaussian blurring. This Myriad2 Harris consumes 116 ms and 0.25 W for the 1 megapixel image (i.e., we still get one order of magnitude slower execution than Zynq). In addition to Harris, we also tested on 12 VLIWs a native/example SW function for Canny edge detection (also not the same with our own Canny benchmark) to derive 25 ms and 1.2 W for a 1 megapixel image. Therefore, compared to ARM Cortex A9 at 667 MHz, which needed 2.8 and 0.5 s for Harris and Canny, respectively, on a 1 megapixel image, we deduce that Myriad2 is 20–25x faster than such an embedded CPU. This factor increases to the area of 30x for a single 11×11 convolution, when Myriad2's time becomes similar even to that of Zynq, which suffers from its PS-PL communication overhead when sending small workloads to PS. Finally, we note that the clock rate on Myriad2 is programmable (e.g., 200–600 MHz), with the overall performance decreasing proportionally and the power consumption decreasing down to 0.5 W.

In addition to high-end DSP, we also test the space-oriented massive parallel processor board (MPPB) [56], which integrates two VLIW Xentium DSPs running at 50 MHz. We ran simulations using the Xentium SDK environment to develop/profile four optimized code versions (specifically fixed/floating-point, C, and assembly) for 2-D convolutions. Notice that these simulations serve only as a best-case scenario because they discard communication delays, as well as LEON, AMBA, and network on chip latency. The final results show that MPPB requires 96–223 ms for one convolution with separable kernels of size 5×5 or 7×7 on a 1024×1024 8-bit image. Such performance is one order of magnitude lower than FPGAs, which are in the area of 10 ms or less. Furthermore, given that our Harris benchmark requires two 5×5 convolutions and three 7×7 convolutions, we estimate that MPPB requires almost 1 s for 1 megapixel image. Hence, currently, it outperforms LEON4 by only 3x, whereas if we make use of multithreading on LEON4, their performances will probably be similar. We note here that MPPB is expected to improve in future versions (e.g., get a 100 MHz clock). However, this is not sufficient to bridge the gap with Myriad2 or FPGAs.

C. Combination and Summary of Results

Table 6 provides a rough comparison of all platforms in terms of performance (normalized throughput), power, and performance per watt, by grouping the platforms in seven categories/columns. Each table cell provides a range of values to capture all the results derived for the designated category (they vary per benchmark and device). In the first row, we normalize the relative performance of all devices such that values around 1 correspond to rad-hard CPUs and values in the area of 1000 correspond to the fastest accelerators. More specifically, we begin with our pair of LEON processors showing $\sim 9\times$ performance gap, on average, and we assign them reference values 0.2 and 1.7, respectively. Subsequently, we consider that Zynq is $\sim 2250\times$ faster than LEON3 for Harris, and we assign a sample value of 450 ($= 0.2 \cdot 2250$). The procedure continues in this fashion until all individual relative performances for all benchmarks have been incorporated in Table 6 (with rounding/simplifications where necessary). Therefore, all results are unified in a linear scale such that the ratio of any two values in Table 6 equals the performance ratio of the selected platforms. The second row reports nominal/measured power, and the third row reports performance per watt. This is the ratio of the first two rows; however, it is calculated separately for each device because the range limits in each category do not necessarily correspond to the same device (e.g., in rad-hard CPUs, throughput = 1.7 represents E698PM, but power = 18 represents RAD5545). Notice that the purpose of Table 6 is to provide succinctly a coarse overview of the vast solution space by focusing, mainly, on the high-performance image processing capabilities of each category. It is intended neither as an exhaustive evaluation of all industrial chips nor as a rigorous comparison between categories. Such a more-accurate comparison is provided in Table 7, for five selected devices, as a result of the second stage of our evaluation methodology.

As shown in Table 6, for single-threaded execution, the space-grade CPUs achieve performance that is comparable to that of the embedded CPUs, albeit consuming more power. Altogether, space-grade and embedded CPUs are 1–2 orders of magnitude slower than desktop CPUs, which however suffer from size limitations and high power consumption and become inappropriate for space applications. Overall, the CPUs achieve the lowest performance per watt ratio by 1–3 orders of magnitude; consequently, they are not deemed as suitable candidates for the application scenario of the current paper. The high-end mobile GPUs overcome the aforementioned power limitations and provide similar performance with desktop CPUs by consuming only ~ 10 W. However, within this low power budget, the high-end DSPs can improve even further the performance per watt of the desktop CPU and mobile GPUs; hence, they should be preferred. The desktop GPUs have great potential for speed but require excessive amounts of power. Compared to mobile GPUs, they trade one order of magnitude of power for speed. Hence, overall, because of relatively low performance per watt or increased power and size, the GPUs are not considered as a good candidate for the application scenario we are concerned with. The FPGAs achieve the highest performance per watt of all platforms by at least one order of magnitude, and even if we include the I/O overhead of the FPGA (e.g., PS-PL communication), they achieve almost the highest throughput that is close to, or in some cases even better than, that of desktop GPUs. Therefore, given the overview of Table 6, we now focus on FPGAs and high-end multicore DSP processors as being the most suitable choices for the space avionics architecture.

Table 7 focuses on one representative FPGA (Xilinx Zynq7045) and two multicore DSP processors (namely 12-core Movidius Myriad2 and eight-core TI 66AK2H14), which for fairness are all SOC devices at 28 nm technology node. In addition, for reference, Table 7 includes one of the latest rad-hard CPUs (OCE E698PM, also SOC) and desktop GPUs (Nvidia GTX 670/680/960, also at 28 nm). The results are based on multiple benchmarks with a fair level of optimization on each platform. Moreover, we highlight that the performance includes all communication overheads (PS-PL for Zynq, memories for Myriad2, etc.), i.e., we evaluate each SOC device as a whole HW/SW system. We note that the throughput of E698PM is hypothetical (i.e., it is extrapolated from our measurements by assuming very efficient four-core parallelization and 600 MHz clock). This represents a best-case scenario for the performance of near-future space-grade CPU, which in practice could prove even $10\times$ lower. For Zynq, the wide throughput range is due to the varying nature of our benchmarks, which result in various acceleration factors versus space-grade CPUs. Specifically, the small factors correspond to feature extraction and large factors to stereo matching. For GPUs, the range is also due to distinct chips, whereas for 66AK2H14, it accounts for diverse results from the literature. According to Table 7, the FPGA achieves $10\times$

Table 6 Coarse comparison of performance (normalized throughput) and power for seven categories of processing platforms (devices summarized in Table 4, e.g., E698PM at 200 MHz, A9 at 667 MHz, etc.; results vary with benchmark and device)

Metric	Rad-hard CPU (single-core)	Embedded CPU (single-core)	Desktop CPU (single-core)	Mobile GPU	High-end DSP	FPGA	Desktop GPU
Throughput	0.2–1.7	0.5–2	20–100	50–150	50–240	300–1460	200–2000
Power, W	1–18	1–2	20–90	6–10	1–10	2–10	70–195
Performance per watt	0.1–0.6	0.25–2	0.5–1	8–15	12–50	60–250	5–25

Table 7 Comparison of performance (normalized throughput) and power for selected SOC at 28 nm technology (excluding rad-hard LEON4); based on multiple benchmarks for images of 1024×1024 8-bit pixels (in-house and literature results)

Metric	Four-core LEON4 (E698PM) 600 MHz	Twelve-core VLIW (Myriad2) 600 MHz	Eight-core DSP (66AK2H14) 1200 MHz	FPGA (Zynq7045) 200–300 MHz	Desktop GPU (GTX 670/680/960) ≥ 1 GHz
Performance	<20	50	70–240	430–1460	600–1800
Power, W	≈ 2.8	1	≈ 10	4–6	>70
Performance per watt	<7	50	7–24	110–240	<25

better performance per watt than all platforms except Myriad2, which is, however, almost 10x slower than FPGA when considering high-definition images and increased FPGA parallelization/resources. If we assume a power budget of 10 W, then the FPGA would be the fastest choice, with 66AK2H14 being 4–6x slower. If we assume an even more limited power budget (e.g., 1–3 W), then Myriad2 improves the CPU throughput by ~3x (or even by one order of magnitude, considering the results of Table 5 instead of the aforementioned extrapolation). Nevertheless, the FPGA could further improve this throughput by 10x by paying only a small penalty in power dissipation. That is, the FPGA can achieve 1–2 orders of magnitude faster execution than any space-grade CPU with comparable power consumption. On the other hand, even with throughput = 20, the CPU will doubtfully meet the increased demands of future vision-based navigation. Altogether, today's multicore DSP architectures approach the FPGA figures in terms of performance and performance per watt, however, not simultaneously for the same device. For instance, we must choose either Myriad2 or 66AK2H14 depending on whether power or speed is the preferred criterion. Moreover, we note that the DSP values of Table 7 are derived with maximum clock frequency (i.e., 1.2 GHz for 66AK2H14). When/if this rate is decreased during space flight (e.g., for reliability reasons), then the speed gap between DSPs and FPGA will widen proportionally; we observe here that the relatively low clock rate is an advantage of FPGAs. Therefore, when both power and performance are important, the FPGA is the most effective architecture/solution currently available.

Beyond performance and power, we consider factors such as the connectivity of the COTS board, radiation tolerance, size/mass, reprogrammability, development effort, and cost. The majority of available COTS boards provide multiple interfaces like Ethernet, USB, PCIe, etc. However, the FPGA boards offer the advantage of extra GPIO and FMC pins, allowing us to communicate with custom connectors and daughterboards. A second advantage of FPGAs relates to the various mitigation techniques that can be applied for error correction due to radiation [92]; these mitigation techniques can be applied almost at gate-level within the same FPGA and, hence, more efficiently than in the case of CPU/core-based processors applying (e.g., triple-core lock step). Considering programmability, the dynamic reconfiguration of SRAM FPGAs renders them almost as useful as the remaining many-core platforms, even for remote updating of their firmware. Considering size/mass, the FPGAs are among the most competitive COTS boards (e.g., $5.72 \times 10.16 \text{ cm}^2$ for the Zynq MMP Zedboard, or $7.8 \times 4.3 \times 1.9 \text{ cm}^3$ for the Zynq Xiphos Q7, with a mass of only 65–24 g). On the downside, as already mentioned, the development effort is increased for FPGAs by a factor of ~4x compared to SW platforms. Still, efficient programming of many-core chips with multiple levels of parallelization also requires an increased amount of effort compared to conventional, serial SW coding.

V. Conclusions

The current paper performed a detailed survey and tradeoff analysis involving an extended number of diverse processing units competing as platforms for space avionics. The set of candidate platforms included both rad-hard and COTS devices, of older and recent technology, such as CPUs, GPUs, multicore DSPs, and FPGAs. Gradually, the present analysis focused on high-performance embedded platforms, which were also compared to more conventional devices for the sake of perspective. The application scenario considered was visual-based navigation, for which a distinct exploration was performed to collect generic specifications and widely used algorithms (i.e., to define a set of representative benchmarks). Overall, the profiling of each device depended both on the benchmark complexity and the underlying HW architecture, and hence, the relative performance of the devices varied greatly among groups and created overlapping clouds of results (with size expanding even by 10x per group). The challenge was tackled in the present comparative study by combining numerous literature results with in-house development/testing, which ultimately led to a big, consistent picture.

The results show that new-generation space-grade CPUs are 10x faster than their predecessors; however, they are still one order of magnitude slower than what will be needed for reliable autonomous VBN. Therefore, high-performance avionics architectures must be designed using HW accelerators. In particular, instead of using multiple chips, it is preferable to use SOC devices, which integrate general-purpose processors and HW accelerators toward size/mass minimization, power reduction, fast intracommunication, reprogrammability, and increased connectivity. Separated by orders of magnitude, the FPGA accelerators provide the highest performance per watt across all platforms, whereas the CPUs provide the lowest, irrespective of their type. In terms of speed alone, high-end desktop GPUs and FPGAs are difficult to distinguish (as groups, they provide similar clouds of results). Likewise, high-end mobile GPUs and many-core DSPs are difficult to distinguish, although the latter have the potential for slightly better performance and power. In terms of speed alone, desktop GPUs and FPGAs are clearly better than mobile GPUs and many-core DSPs. In terms of power, desktop CPUs and GPUs seem prohibitive for space avionics; however, a 10 W budget is enough to allow many-core DSPs, mobile GPUs, or FPGAs to accelerate a conventional space-grade CPU by 1–3 orders of magnitude. In such a high-performance embedded computing scenario, with relaxed constraints on radiation tolerance due to mission specifications, it would be preferable to use COTS 28 nm SOC FPGAs, which provide 2–29x faster execution than 28 nm DSP processors.

Acknowledgments

This work has received support from the ESA via the High Performance Avionics Solution for Advanced and Complex GNC Systems project (European Space Research and Technology Centre, ESA, ref. 4000117700/16/NL/LF).

References

- [1] Bonnal, C., Ruault, J.-M., and Desjean, M.-C., "Active Debris Removal: Recent Progress and Current Trends," *Acta Astronautica*, Vol. 85, April–May 2013, pp. 51–60.
doi:10.1016/j.actaastro.2012.11.009
- [2] Fortescue, P., Swinerd, G., and Stark, J., *Spacecraft Systems Engineering*, Wiley, West Sussex, U.K., 2011, p. 752.
- [3] Pignol, M., "DMT and DT2: Two Fault-Tolerant Architectures Developed by CNES for COTS-Based Spacecraft Supercomputers," *Proceedings of the International On-Line Testing Symposium*, IEEE Publ., 2006, p. 10.
- [4] Pignol, M., "COTS-Based Applications in Space Avionics," *Proceedings of the Conference on Design, Automation and Test in Europe*, IEEE Publ., Piscataway, NJ, 2010, pp. 1213–1219.
- [5] Selva, D., and Krejci, D., "A Survey and Assessment of the Capabilities of Cubesats for Earth Observation," *Acta Astronautica*, Vol. 74, May–June 2012, pp. 50–68.
doi:10.1016/j.actaastro.2011.12.014
- [6] Furano, G., and Menicucci, A., "Roadmap for On-Board Processing and Data Handling Systems in Space," *Dependable Multicore Architectures at Nanoscale*, edited by M. Ottavi, D. Gizopoulos, and S. Pontarelli, Springer, Cham, Switzerland, 2018, pp. 253–281.
- [7] Maurer, R. H., Fraeman, M. E., Martin, M. N., and Roth, D. R., "Harsh Environments: Space Radiation Environment, Effects, and Mitigation," *Johns Hopkins APL Technical Digest*, Vol. 28, No. 1, 2008, pp. 17–29.
- [8] Geller, D., "Orbital Rendezvous: When Is Autonomy Required?" *Journal of Guidance, Control, and Dynamics*, Vol. 30, No. 4, 2007, pp. 974–981.
doi:10.2514/1.27052

- [9] "GSTP Element 1 'Develop'—Compendium of Potential Activities 2017: Clean Space," European Space Agency Rept. ESA-GSTP-TECT-PL-005452, Noordwijk, The Netherlands, May 2017.
- [10] Liou, J.-C., "Engineering and Technology Challenges for Active Debris Removal," *EUCASS Proceedings Series*, Vol. 4, March 2013, pp. 735–748.
- [11] Flores-Abad, A., Ma, O., Pham, K., and Ulrich, S., "A Review of Space Robotics Technologies for On-Orbit Servicing," *Progress in Aerospace Sciences*, Vol. 68, July 2014, pp. 1–26.
doi:10.1016/j.paerosci.2014.03.002
- [12] Pinto, R., Berrojo, L., Garcia, E., Trautner, R., Rauwerda, G., Sunesen, K., Redant, S., Habinc, S., Andersson, J., and López, J., "Scalable Sensor Data Processor: Architecture and Development Status," *Proceedings of DSP Day 2016*, European Space Agency, Gothenburg, Sweden, 2016, <https://indico.esa.int/indico/event/102/session/22/contribution/35/material/slides/0.pdf> [retrieved 30 March 2017].
- [13] Lepape, E., "NanoXplore NXT-32000 FPGA (NG-MEDIUM) Presentation," *Proceedings of the 3rd SEF/UW Workshop*, ESTEC, ESA, 2016, <https://indico.esa.int/indico/event/130/session/11/contribution/59/material/slides/0.pdf> [retrieved 30 March 2017].
- [14] Biesbroek, R., Innocenti, L., Wolahan, A., and Serrano, S. M., "e.Deorbit—ESA's Active Debris Removal Mission," *Proceedings of the 7th European Conference on Space Debris*, ESA Space Debris Office, 2017, p. 10.
- [15] Tweddle, B. E., "Computer Vision-Based Localization and Mapping of an Unknown, Uncooperative and Spinning Target for Spacecraft Proximity Operations," Ph.D. Thesis, Dept. of Aeronautics and Astronautics, Massachusetts Inst. of Technology, Cambridge, MA, 2013.
- [16] Woffinden, D., and Geller, D., "Navigating the Road to Autonomous Orbital Rendezvous," *Journal of Spacecraft and Rockets*, Vol. 44, No. 4, 2007, pp. 898–909.
doi:10.2514/1.30734
- [17] Lentaris, G., Stamoulis, I., Soudris, D., and Lourakis, M., "HW/SW Codesign and FPGA Acceleration of Visual Odometry Algorithms for Rover Navigation on Mars," *IEEE Transactions on Circuits and Systems for Video Technology*, Vol. 26, No. 8, Aug. 2016, pp. 1563–1577.
doi:10.1109/TCSVT.2015.2452781
- [18] Matthies, L., Maimone, M., Johnson, A., Cheng, Y., Willson, R., Villalpando, C., Goldberg, S., Huertas, A., Stein, A., and Angelova, A., "Computer Vision on Mars," *International Journal of Computer Vision*, Vol. 75, No. 1, 2007, pp. 67–92.
doi:10.1007/s11263-007-0046-z
- [19] Van Pham, B., Lacroix, S., and Devy, M., "Vision-Based Absolute Navigation for Descent and Landing," *Journal of Field Robotics*, Vol. 29, No. 4, July 2012, pp. 627–647.
doi:10.1002/rob.21406
- [20] Delaune, J., Besnerais, G. L., Voirin, T., Farges, J., and Bourdarias, C., "Visual-Inertial Navigation for Pinpoint Planetary Landing Using Scale-Based Landmark Matching," *Robotics and Autonomous Systems*, Vol. 78, April 2016, pp. 63–82.
- [21] Lourakis, M., and Zabulis, X., "Model-Based Visual Tracking of Orbiting Satellites Using Edges," *Proceedings of the IEEE/RSJ/IGI International Conference on Intelligent Robots and Systems*, IEEE Publ., Piscataway, NJ, 2017, pp. 3791–3796.
- [22] English, C., Okounova, G., Saint-Cyr, P., Choudhuri, A., and Luu, T., "Real-Time Dynamic Pose Estimation Systems in Space: Lessons Learned for System Design and Performance Evaluation," *International Journal of Intelligent Control and Systems*, Vol. 16, No. 2, 2011, pp. 79–96.
- [23] Lepetit, V., and Fua, P., "Monocular Model-Based 3D Tracking of Rigid Objects: A Survey," *Foundations and Trends in Computer Graphics and Vision*, Vol. 1, No. 1, 2005, pp. 1–89.
doi:10.1561/06000000001
- [24] Petit, A., "Robust Visual Detection and Tracking of Complex Objects: Applications to Space Autonomous Rendezvous and Proximity Operations," Ph.D. Thesis, Université Rennes 1, Rennes, France, 2013.
- [25] Lourakis, M., and Zabulis, X., "Accurate Scale Factor Estimation in 3D Reconstruction," *Proceedings of the International Conference on Computer Analysis of Images and Patterns*, Springer, Berlin, 2013, pp. 498–506.
- [26] Mikolajczyk, K., Tuytelaars, T., Schmid, C., Zisserman, A., Matas, J., Schaffalitzky, F., Kadir, T., and Gool, L. V., "A Comparison of Affine Region Detectors," *International Journal of Computer Vision*, Vol. 65, Nos. 1–2, Nov. 2005, pp. 43–72.
doi:10.1007/s11263-005-3848-x
- [27] Canny, J., "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 8, No. 6, Nov. 1986, pp. 679–698.
doi:10.1109/TPAMI.1986.4767851
- [28] Harris, C., and Stephens, M., "A Combined Corner and Edge Detector," *Proceedings of the 4th Alvey Vision Conference*, Alvey Vision Club, U.K., 1988, pp. 147–151.
- [29] Vacchetti, L., Lepetit, V., and Fua, P., "Stable Real-Time 3D Tracking Using Online and Offline Information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 26, No. 10, 2004, pp. 1385–1391.
doi:10.1109/TPAMI.2004.92
- [30] Collet Romea, A., and Srinivasa, S., "Efficient Multi-View Object Recognition and Full Pose Estimation," *Proceedings of the IEEE International Conference on Robotics and Automation*, IEEE Publ., Piscataway, NJ, May 2010, pp. 2050–2055.
- [31] Augenstein, S., "Monocular Pose and Shape Estimation of Moving Targets for Autonomous Rendezvous and Docking," Ph.D. Thesis, Dept. of Aeronautics and Astronautics, Stanford Univ., Palo Alto, CA, 2011.
- [32] Lourakis, M., and Zabulis, X., "Model-Based Pose Estimation for Rigid Objects," *Proceedings of the International Conference on Computer Vision Systems*, Springer-Verlag, Berlin, 2013, pp. 83–92.
- [33] Drummond, T., and Cipolla, R., "Real-Time Visual Tracking of Complex Structures," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 24, No. 7, 2002, pp. 932–946.
doi:10.1109/TPAMI.2002.1017620
- [34] Comport, A. I., Marchand, É., Pressigout, M., and Chaumette, F., "Real-Time Markerless Tracking for Augmented Reality: The Virtual Visual Servoing Framework," *IEEE Transactions on Visualization and Computer Graphics*, Vol. 12, No. 4, 2006, pp. 615–628.
doi:10.1109/TVCG.2006.78
- [35] Petit, A., Marchand, E., and Kanani, K., "Tracking Complex Targets for Space Rendezvous and Debris Removal Applications," *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE Publ., Piscataway, NJ, 2012, pp. 4483–4488.
- [36] D'Amico, S., Benn, M., and Jørgensen, J., "Pose Estimation of an Uncooperative Spacecraft from Actual Space Imagery," *International Journal of Space Science and Engineering*, Vol. 2, No. 2, 2014, pp. 171–189.
doi:10.1504/IJSPACESE.2014.060600
- [37] Vacchetti, L., Lepetit, V., and Fua, P., "Combining Edge and Texture Information for Real-Time Accurate 3D Camera Tracking," *Proceedings of the International Symposium on Mixed and Augmented Reality*, IEEE Publ., Piscataway, NJ, 2004, pp. 48–57.
- [38] Rosten, E., and Drummond, T., "Fusing Points and Lines for High Performance Tracking," *Proceedings of the IEEE International Conference on Computer Vision*, Vol. 2, IEEE Publ., Piscataway, NJ, 2005, pp. 1508–1515.
- [39] Choi, C., and Christensen, H. I., "Real-Time 3D Model-Based Tracking Using Edge and Keypoint Features for Robotic Manipulation," *Proceedings of the IEEE International Conference on Robotics and Automation*, IEEE Publ., Piscataway, NJ, 2010, pp. 4048–4055.
- [40] Augenstein, S., and Rock, S. M., "Improved Frame-to-Frame Pose Tracking During Vision-Only SLAM/SFM with a Tumbling Target," *Proceedings of the IEEE International Conference on Robotics and Automation*, IEEE Publ., Piscataway, NJ, 2011, pp. 3131–3138.
- [41] Tweddle, B. E., Saenz-Otero, A., Leonard, J. J., and Miller, D. W., "Factor Graph Modeling of Rigid-Body Dynamics for Localization, Mapping, and Parameter Estimation of a Spinning Object in Space," *Journal of Field Robotics*, Vol. 32, No. 6, 2015, pp. 897–933.
doi:10.1002/rob.21548

- [42] Post, M. A., Li, J., and Clark, C., "Visual Pose Estimation System for Autonomous Rendezvous of Spacecraft," *Proceedings of the Symposium on Advanced Space Technologies in Automation and Robotics*, Strathclyde, U.K., 2015, p. 9.
- [43] Scharstein, D., and Szeliski, R., "A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms," *International Journal of Computer Vision*, Vol. 47, Nos. 1–3, 2002, pp. 7–42.
doi:10.1023/A:1014573219977
- [44] Zabulis, X., Lourakis, M., and Koutlemanis, P., "Correspondence-Free Pose Estimation for 3D Objects from Noisy Depth Data," *Visual Computer*, Vol. 34, No. 2, Feb. 2018, pp. 193–211.
doi:10.1007/s00371-016-1326-9
- [45] Jasiedzki, P., Greenspan, M., and Roth, G., "Pose Determination and Tracking for Autonomous Satellite Capture," *Proceedings of the International Symposium on Artificial Intelligence and Robotics and Automation in Space*, National Research Council of Canada, 2001, p. 8.
- [46] Yu, F., He, Z., Qiao, B., and Yu, X., "Stereo-Vision-Based Relative Pose Estimation for the Rendezvous and Docking of Noncooperative Satellites," *Mathematical Problems in Engineering*, Vol. 2014, No. 11, 2014, pp. 1–12.
- [47] Li, R., Zhou, Y., Chen, F., and Chen, Y., "Parallel Vision-Based Pose Estimation for Non-Cooperative Spacecraft," *Advances in Mechanical Engineering*, Vol. 7, No. 7, 2015, pp. 1–9.
doi:10.1177/1687814015594312
- [48] Carlo, S. D., Prinetto, P., Rolfo, D., Sansonne, N., and Trotta, P., "A Novel Algorithm and Hardware Architecture for Fast Video-Based Shape Reconstruction of Space Debris," *EURASIP Journal on Advances in Signal Processing*, Vol. 2014, No. 1, 2014, p. 147.
doi:10.1186/1687-6180-2014-147
- [49] Xu, W., Liang, B., Li, C., and Xu, Y., "Autonomous Rendezvous and Robotic Capturing of Non-Cooperative Target in Space," *Robotica*, Vol. 28, No. 5, 2010, pp. 705–718.
doi:10.1017/S0263574709990397
- [50] Xu, W., Xue, Q., Liu, H., Du, X., and Liang, B., "A Pose Measurement Method of a Non-Cooperative GEO Spacecraft Based on Stereo Vision," *Proceedings of the International Conference on Control Automation Robotics Vision*, IEEE Publ., Piscataway, NJ, 2012, pp. 966–971.
- [51] Hillman, R., Swift, G., Layton, P., Conrad, M., Thibodeau, C., and Irom, F., "Space Processor Radiation Mitigation and Validation Techniques for an 1,800 MIPS Processor Board," *Proceedings of the 7th European Conference on Radiation and Its Effects on Components and Systems*, IEEE Publ., Piscataway, NJ, 2003, pp. 347–352.
- [52] Notebaert, O., "Research & Technology Activities in On-Board Data Processing Domain," *Avionics Technology Trends Workshop*, ESTEC, ESA, Oct. 2015, <https://indico.esa.int/indico/event/85/session/12/contribution/96/material/0/0.pdf> [retrieved 30 March 2017].
- [53] "High Reliability Processing Node (HRPN-LEON4)," Airbus Defence and Space, Bremen, Germany, 2015, <http://microelectronics.esa.int/gr740/Airbus-ST-NGMP-Board-Flyer-HRPN.pdf> [retrieved 30 March 2017].
- [54] Stureson, F., Gaisler, J., Ginosar, R., and Liran, T., "Radiation Characterization of a Dual Core LEON3-FT Processor," *Proceedings of the 12th European Conference on Radiation and Its Effects on Components and Systems*, IEEE Publ., Piscataway, NJ, 2011, pp. 938–944.
- [55] Guertin, S. M., "CubeSat and Mobile Processors," *NASA Electronics Technology Workshop*, 2015, pp. 23–26, <https://npp.nasa.gov/workshops/etw2015/talks.cfm>.
- [56] Pinto, R., Sanchez, P., Berrojo, L., Sunesen, K., Potman, J., and Rauwerda, G., "On-Board Compression Using a Next-Generation Multicore DSP Architecture," *Proceedings of the 5th International Workshop on On-Board Payload Data Compression [CDROM]*, ESA Conference Bureau, ESA ESRIN, Frascati, Italy, 2016.
- [57] Iturbe, X., Keymeulen, D., Ozer, E., Yiu, P., Berisford, D., Hand, K., and Carlson, R., "An Integrated SoC for Science Data Processing in Next-Generation Space Flight Instruments Avionics," *Proceedings of the 2015 IFIP/IEEE International Conference on Very Large Scale Integration*, IEEE Publ., Piscataway, NJ, 2015, pp. 134–141.
- [58] Rudolph, D., Wilson, C., Stewart, J., Gauvin, P., George, A., Lam, H., Crum, G. A., Wirthlin, M., Wilson, A., and Stoddard, A., "CSP: A Multifaceted Hybrid Architecture for Space Computing," *Proceedings of the 28th Annual AIAA/USU Conference on Small Satellites*, AIAA, Reston, VA, 2014, p. 8.
- [59] Flatley, T. P., "SpaceCube: A Family of Reconfigurable Hybrid On-Board Science Data Processors," *Proceedings of the 2014 International Conference on ReConfigurable Computing and FPGAs*, IEEE Publ., Piscataway, NJ, 2014, p. 1.
- [60] Petrick, D., Gill, N., Hassoun, M., Stone, R., Winternitz, L., Thomas, L., Davis, M., Sparacino, P., and Flatley, T., "Adapting the SpaceCube v2.0 Data Processing System for Mission-Unique Application Requirements," *Proceedings of the 2015 NASA/ESA Conference on Adaptive Hardware and Systems (AHS)*, IEEE Publ., Piscataway, NJ, 2015, pp. 1–8.
- [61] Nah, J.-H., Suh, Y., and Lim, Y., "L-Bench: An Android Benchmark Set for Low-Power Mobile GPUs," *Computers & Graphics*, Vol. 61, Dec. 2016, pp. 40–49.
doi:10.1016/j.cag.2016.09.002
- [62] Gao, Y., Zhang, F., and Bakos, J. D., "Sparse Matrix-Vector Multiply on the Keystone II Digital Signal Processor," *Proceedings of the 2014 IEEE High Performance Extreme Computing Conference*, IEEE Publ., Piscataway, NJ, 2014, pp. 1–6.
- [63] Han, S., Mao, H., and Dally, W. J., "Deep Compression: Compressing Deep Neural Network with Pruning, Trained Quantization and Huffman Coding," *CoRR*, abs/1510.00149, Vol. 2, 2015.
- [64] Kim, S. C., and Bhattacharyya, S. S., "An Efficient GPU Implementation of a Multirate Resampler for Multi-Carrier Systems," *Proceedings of the 2015 IEEE Global Conference on Signal and Information Processing*, IEEE Publ., Piscataway, NJ, 2015, pp. 751–755.
- [65] Mitra, G., Johnston, B., Rendell, A. P., McCreath, E., and Zhou, J., "Use of SIMD Vector Operations to Accelerate Application Code Performance on Low-Powered ARM and Intel Platforms," *Proceedings of the 27th IEEE International Parallel and Distributed Processing Symposium Workshops & PhD Forum*, IEEE Publ., Piscataway, NJ, 2013, pp. 1107–1116.
- [66] Peng, B., Wang, T., Jin, X., and Wang, C., "An Accelerating Solution for N-Body MOND Simulation with FPGA-SoC," *International Journal of Reconfigurable Computing*, Vol. 2016, 2016, pp. 1–10.
doi:10.1155/2016/4592780
- [67] Cloutier, M. F., Paradis, C., and Weaver, V. M., "A Raspberry Pi Cluster Instrumented for Fine-Grained Power Measurement," *Electronics*, Vol. 5, No. 4, 2016, p. 61.
doi:10.3390/electronics5040061
- [68] Sioutas, T., Corporaal, H., van den Braak, G.-J., and Larisis, N., "Benchmarking Novel Multi-SoC DSP Architecture," B.S. Thesis, Eindhoven Univ. of Technology, Eindhoven, The Netherlands, Nov. 2015.
- [69] Kumar, V., Shiplea, A., Jayaraj, A., Budimlić, Z., Majeti, D., and Sarkar, V., "Heterogeneous Work-Stealing Across CPU and DSP Cores," *Proceedings of the 2015 IEEE High Performance Extreme Computing Conference*, IEEE Publ., Piscataway, NJ, 2015, pp. 1–6.
- [70] Castillo, M. I., Fernández, J. C., Igual, F. D., Plaza, A., Quintana-Ortí, E. S., and Remón, A., "Hyperspectral Unmixing on Multicore DSPs: Trading Off Performance for Energy," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, Vol. 7, No. 6, 2014, pp. 2297–2304.
doi:10.1109/JSTARS.2013.2266927
- [71] Menant, J., Pressigout, M., Morin, L., and Nezan, J.-F., "Optimized Fixed Point Implementation of a Local Stereo Matching Algorithm onto C66x DSP," *Proceedings of the Conference on Design and Architectures for Signal and Image Processing*, IEEE Publ., Piscataway, NJ, 2014, pp. 1–6.
- [72] Lentaris, G., Diamantopoulos, D., Siozios, K., Soudris, D., and Rodríguez, M. A., "Hardware Implementation of Stereo Correspondence Algorithm for the Exomars Mission," *Proceedings of the 22nd International Conference on Field Programmable Logic and Applications*, IEEE Publ., Piscataway, NJ, 2012, pp. 667–670.
- [73] Pelcat, M., Desnos, K., Heulot, J., Guy, C., Nezan, J.-F., and Aridhi, S., "Preesm: A Dataflow-Based Rapid Prototyping Framework for Simplifying Multicore DSP Programming," *Proceedings of the European Embedded Design in Education and Research Conference*, IEEE Publ., Piscataway, NJ, 2014, pp. 36–40.
- [74] Coombs, J., and Prabhu, R., "OpenCV on TI's DSP+ARM Platforms: Mitigating the Challenges of Porting OpenCV to Embedded Platforms," Texas Instruments Inc. Rept. SPRY175, Dallas, TX, 2011.

- [75] Ramesh, B., Bhardwaj, A., Richardson, J., George, A. D., and Lam, H., "Optimization and Evaluation of Image-and Signal-Processing Kernels on the TI C6678 Multi-Core DSP," *Proceedings of the High Performance Extreme Computing Conference*, IEEE Publ., Piscataway, NJ, 2014, pp. 1–6.
- [76] Ginosar, R., Aviely, P., Gellis, H., Liran, T., Israeli, T., Neshet, R., Lange, F., Dobkin, R., Meirov, H., and Reznik, D., "RC64, a Rad-Hard Many-Core High-Performance DSP for Space Applications," *European Space Agency*, Vol. 732, Special Publ., Noordwijk, The Netherlands, 2015, p. 5.
- [77] Tippetts, B., Lee, D. J., Lillywhite, K., and Archibald, J., "Review of Stereo Vision Algorithms and Their Suitability for Resource-Limited Systems," *Journal of Real-Time Image Processing*, Vol. 11, No. 1, 2016, pp. 5–25.
doi:10.1007/s11554-012-0313-2
- [78] Chen, J., Alfred, C., and So, H. K.-H., "Design Considerations of Real-Time Adaptive Beamformer for Medical Ultrasound Research Using FPGA and GPU," *Proceedings of the International Conference on Field-Programmable Technology*, IEEE Publ., Piscataway, NJ, 2012, pp. 198–205.
- [79] Pauwels, K., Tomasi, M., Alonso, J. D., Ros, E., and Van Hulle, M. M., "A Comparison of FPGA and GPU for Real-Time Phase-Based Optical Flow, Stereo, and Local Image Features," *IEEE Transactions on Computers*, Vol. 61, No. 7, 2012, pp. 999–1012.
doi:10.1109/TC.2011.120
- [80] Xu, Q., Varadarajan, S., Chakrabarti, C., and Karam, L. J., "A Distributed Canny Edge Detector: Algorithm and FPGA Implementation," *IEEE Transactions on Image Processing*, Vol. 23, No. 7, 2014, pp. 2944–2960.
doi:10.1109/TIP.2014.2311656
- [81] Tweddle, B. E., "Computer Vision Based Navigation for Spacecraft Proximity Operations," M.S. Thesis, Dept. of Aeronautics and Astronautics, Massachusetts Inst. of Technology, Cambridge, MA, 2010.
- [82] Vellas, S., Lentaris, G., Maragos, K., Soudris, D., Kandylakis, Z., and Karantzalos, K., "FPGA Acceleration of Hyperspectral Image Processing for High-Speed Detection Applications," *Proceedings of the 2017 IEEE International Symposium on Circuits and Systems*, IEEE Publ., Piscataway, NJ, 2017, pp. 1–4.
- [83] Rubner, Y., Puzicha, J., Tomasi, C., and Buhmann, J., "Empirical Evaluation of Dissimilarity Measures for Color and Texture," *Computer Vision and Image Understanding*, Vol. 84, No. 1, 2001, pp. 25–43.
doi:10.1006/cviu.2001.0934
- [84] Horn, B., "Closed-Form Solution of Absolute Orientation Using Unit Quaternions," *Journal of the Optical Society of America A*, Vol. 4, No. 4, April 1987, pp. 629–642.
doi:10.1364/JOSAA.4.000629
- [85] "Intel Quark SoC X1000 Core Hardware Reference Manual," Intel, Santa Clara, CA, Oct. 2013.
- [86] Crockett, L. H., Elliot, R. A., Enderwitz, M. A., and Stewart, R. W., *The Zynq Book: Embedded Processing with the Arm Cortex-A9 on the Xilinx Zynq-7000 All Programmable SoC*, Strathclyde Academic Media, U.K., 2014, p. 484.
- [87] Lentaris, G., Stamoulias, I., Diamantopoulos, D., Maragos, K., Siozios, K., Soudris, D., Rodríguez, M. A., Lourakis, M., Zabulis, X., Kostavelis, I., Nalpantidis, L., Boukas, E., and Gasteratos, A., "SPARTAN/SEXTANT/COMPASS: Advancing Space Rover Vision via Reconfigurable Platforms," *Proceedings of the International Symposium on Applied Reconfigurable Computing*, Springer, Cham, 2015, pp. 475–486.
- [88] "Xillybus FPGA Designer's Guide, Version 2.0," Xillybus Ltd., Haifa, Israel, 2016, http://xillybus.com/downloads/doc/xillybus_fpga_api.pdf [retrieved 30 March 2017].
- [89] Georgis, G., Lentaris, G., and Reisis, D., "Acceleration Techniques and Evaluation on Multi-Core CPU, GPU and FPGA for Image Processing and Super-Resolution," *Journal of Real-Time Image Processing*, July 2016, pp. 1–28.
- [90] Lucero, J., and Slous, B., "Designing High-Performance Video Systems with the Zynq-7000 All Programmable SoC Using IP Integrator," *Xilinx Application Note XAPP1205 (v1.0)*, Rept. XAPP1205, Xilinx, Santa Clara, CA, March 2014, https://www.xilinx.com/support/documentation/application_notes/xapp1205-high-performance-video-zynq.pdf.
- [91] Barry, B., Brick, C., Connor, F., Donohoe, D., Moloney, D., Richmond, R., O'Riordan, M., and Toma, V., "Always-on Vision Processing Unit for Mobile Applications," *IEEE Micro*, Vol. 35, No. 2, 2015, pp. 56–66.
doi:10.1109/MM.2015.10
- [92] Siegle, F., Vladimirova, T., Ilstad, J., and Emam, O., "Mitigation of Radiation Effects in SRAM-Based FPGAs for Space Applications," *ACM Computing Surveys*, Vol. 47, No. 2, Jan. 2015, pp. 1–34.
doi:10.1145/2658850

E. W. Frew
Associate Editor